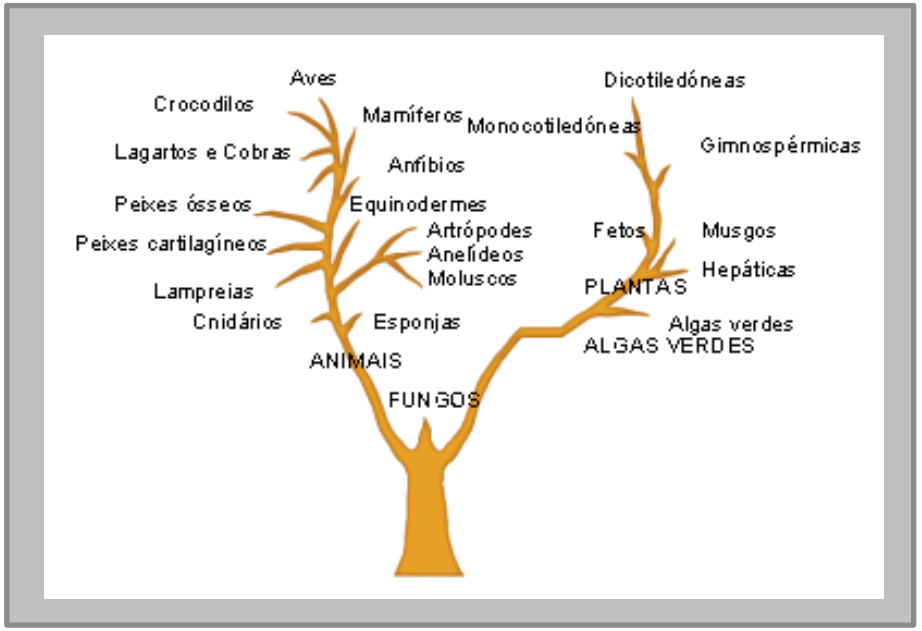
Árvores



Árvores

Em muitas aplicações, estruturas lineares não são as mais adequadas para a representação dos dados.



As árvores são estruturas que nos permitem desenvolver algoritmos mais eficientes.

Definição

Uma árvore enraizada, *T*, é um conjunto finito de elementos chamados vértices (ou nós), tal que:

i) ***T* = ∅ (a árvore é dita vazia) ou**

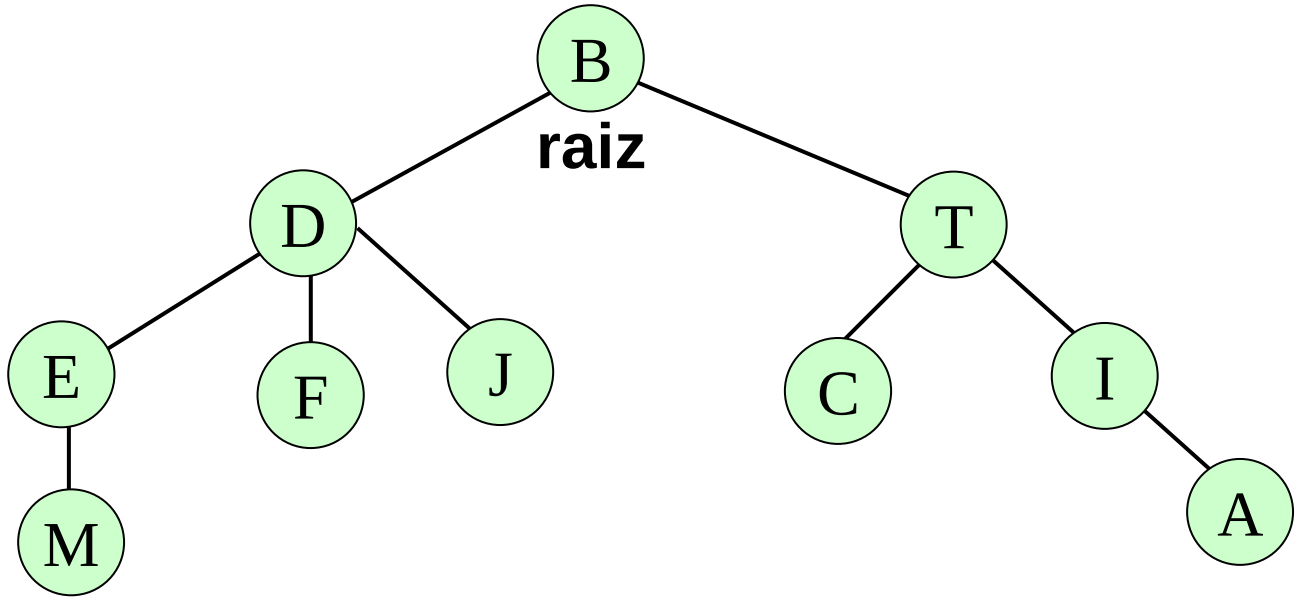
ii) Existe um nó especial, *r*, dito *raiz* da árvore *T* e os nós restantes constituem um único conjunto vazio ou são divididos em *m* ≥ 1 conjuntos disjuntos não vazios, chamados de *subárvores de r*, cada qual uma árvore.



Cajueiro de Pirangi

Representação

A representação mais comum é a hierárquica.



Representação

Que outras representações existem para árvores além da hierárquica?



Representação

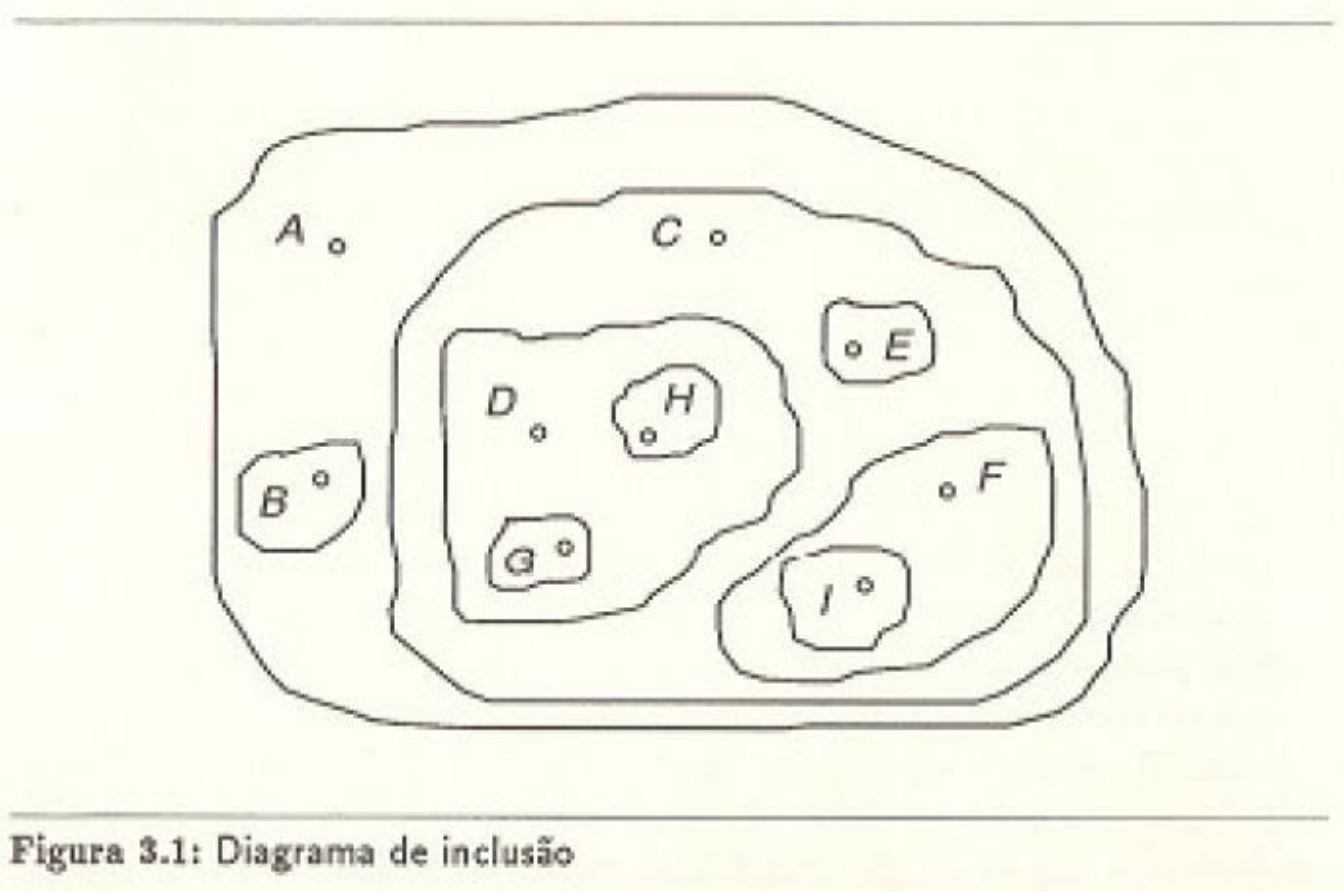
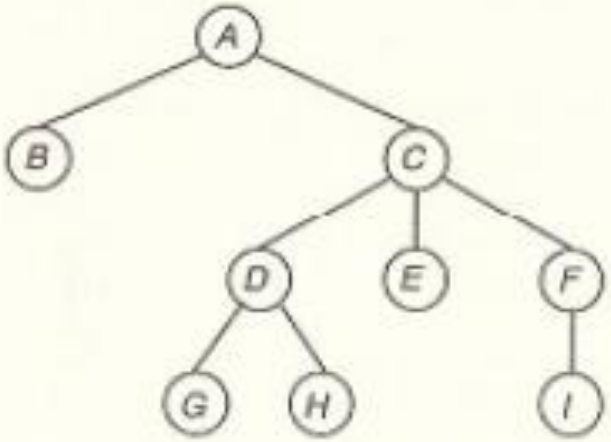
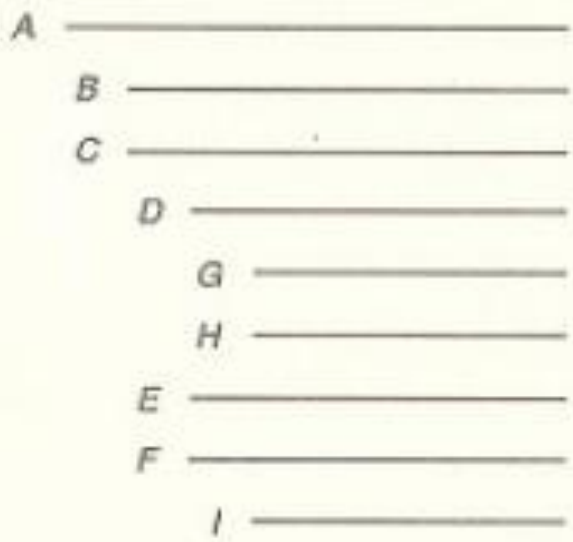


Figura 3.1: Diagrama de inclusão



(a)



(b)

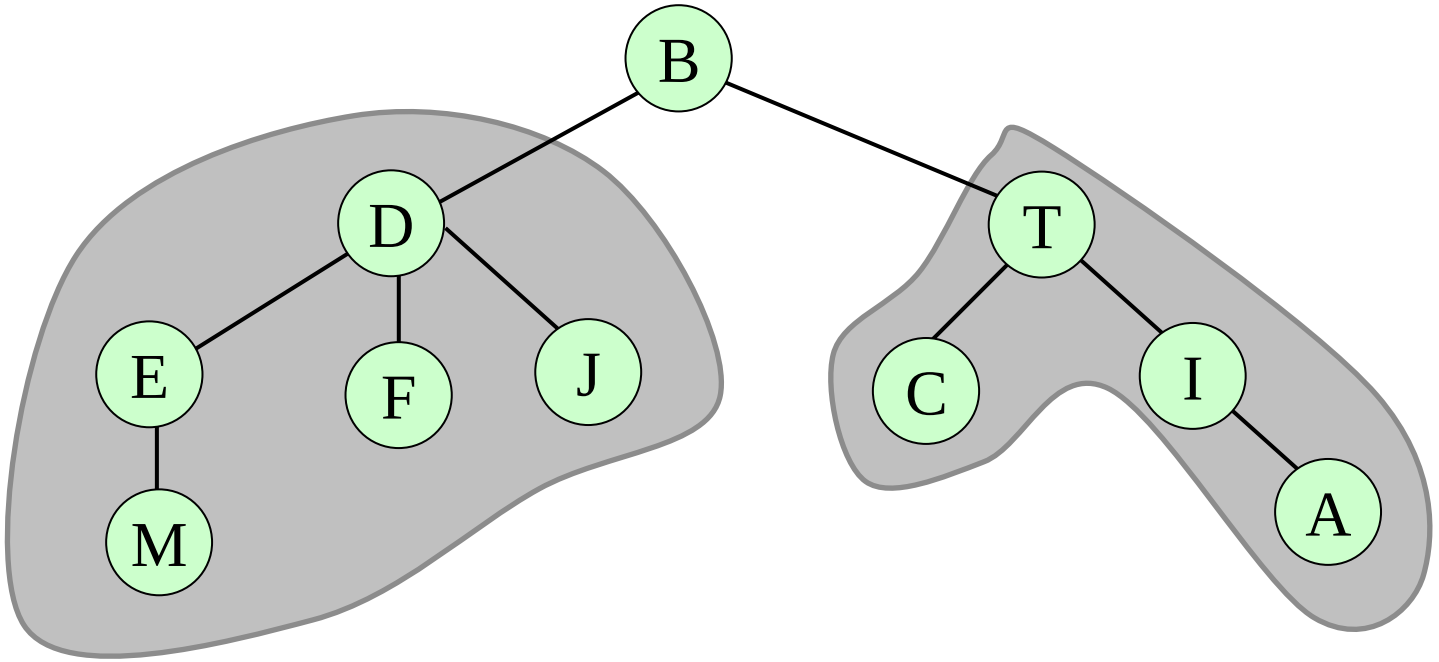
(c)

(A (B) (C (D (G) (H)) (E) (F (I))))

Figura 3.2: Representações de uma árvore

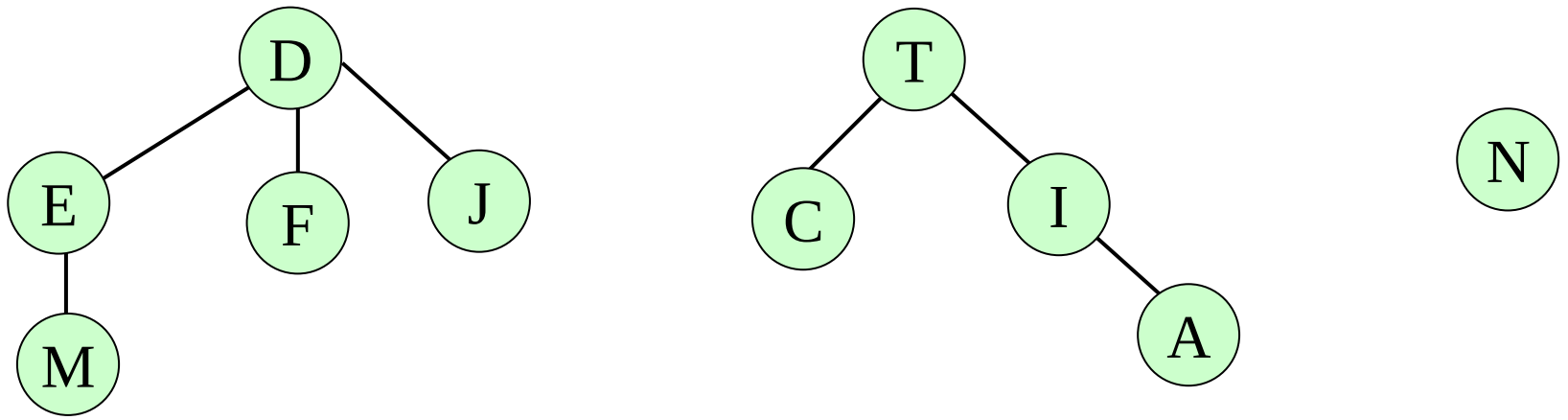
Definições

Subárvores da raiz



Definições

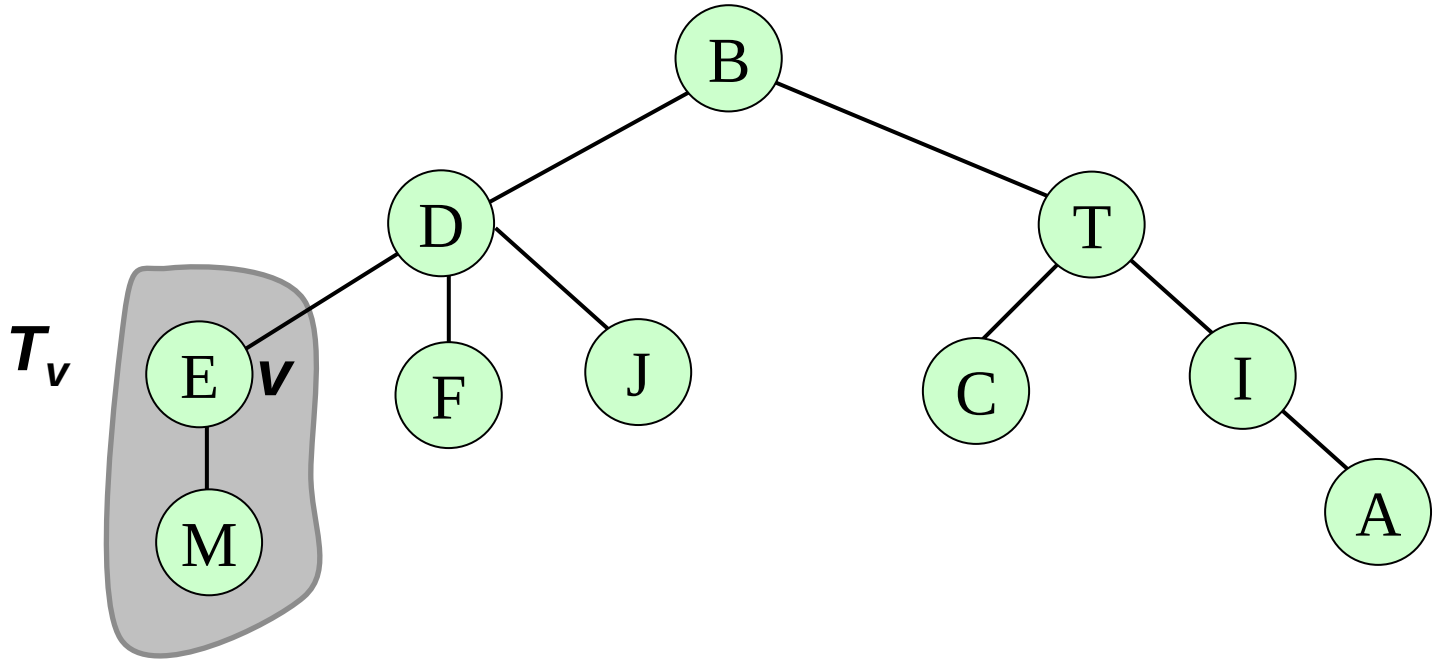
Floresta = um conjunto de árvores



Árvore 1 **Árvore 2** **Árvore 3**

Definições

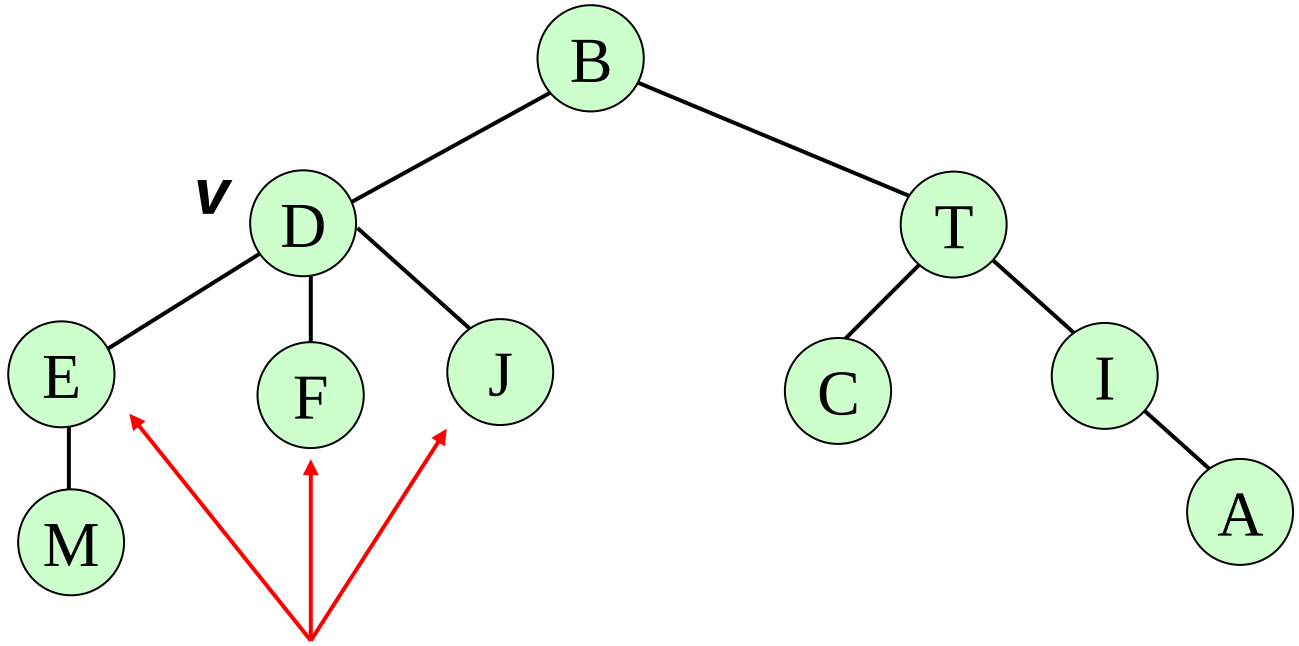
Seja *v* um nó de *T*, a notação T_v representa a subárvore de *T* com raiz *v*.



Árvore *T*

Definições

Seja *v* o nó raiz de T_v .
Os nós raízes $w_1,...,w_j$ das subárvores de T_v são os *filhos* de *v*.



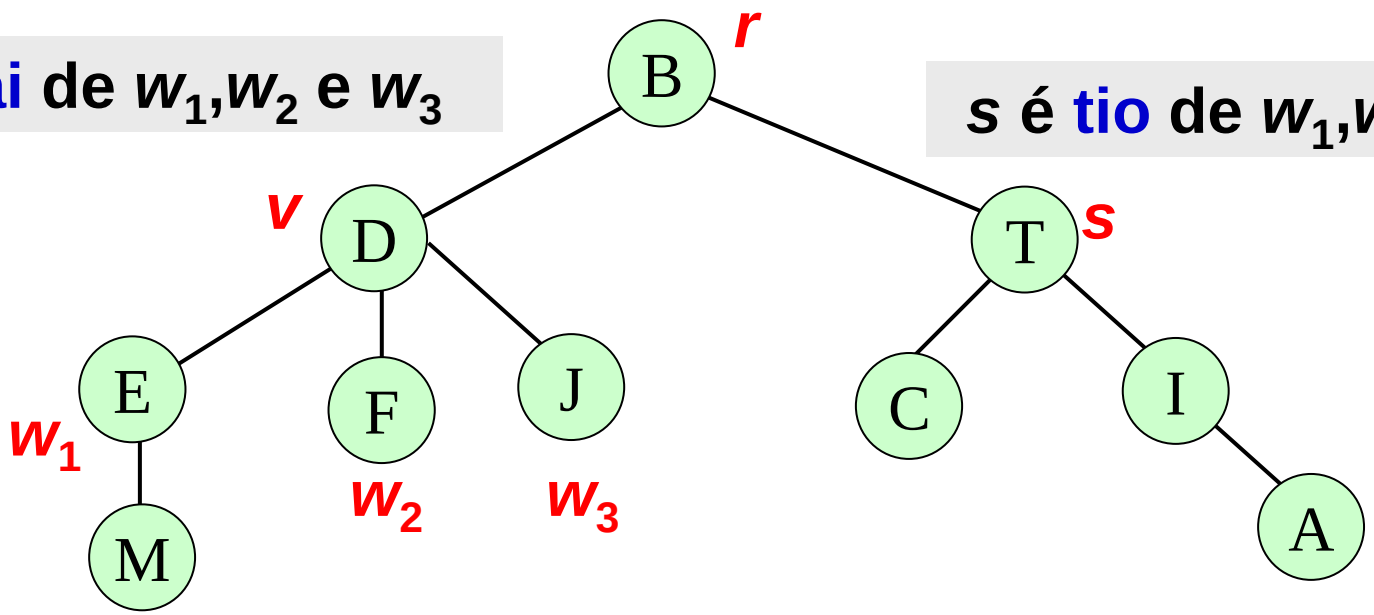
Filhos de *v*

Definições

r é **avô** de w_1,w_2 e w_3

v é **pai** de w_1,w_2 e w_3

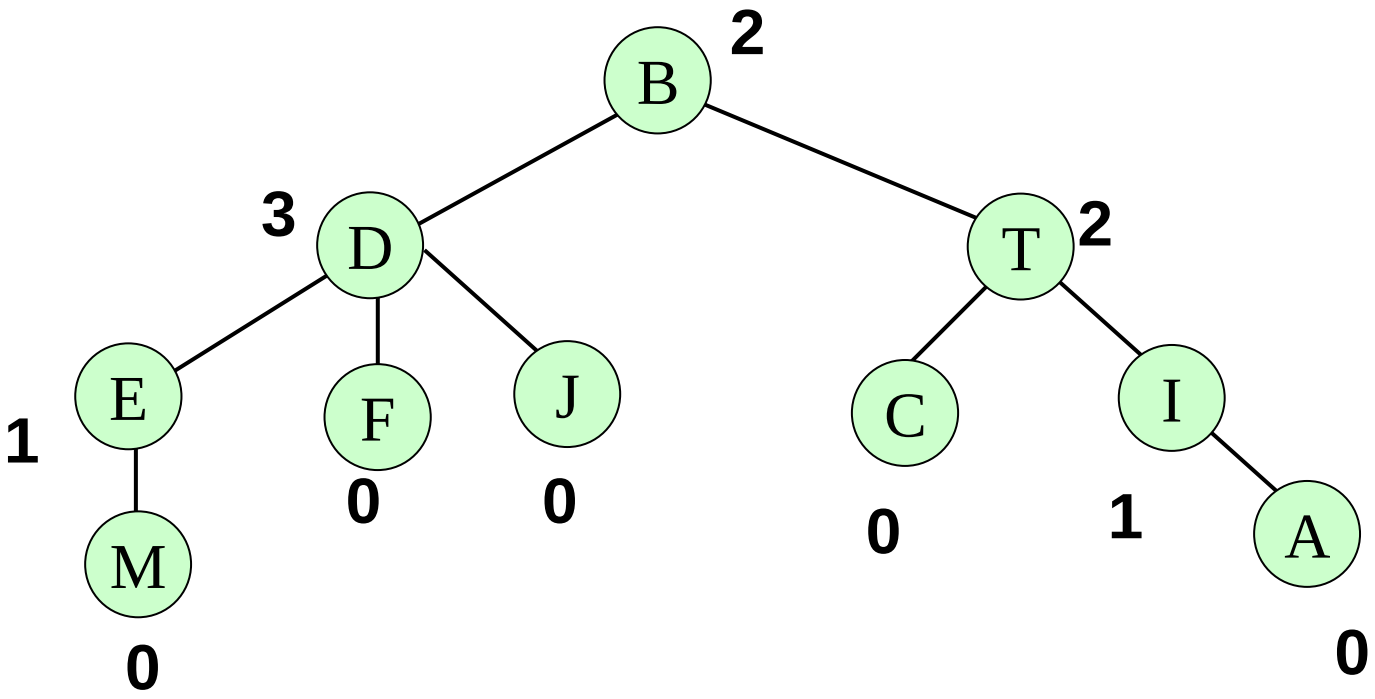
s é **tio** de w_1,w_2 e w_3



w_1,w_2 e w_3 são **irmãos**

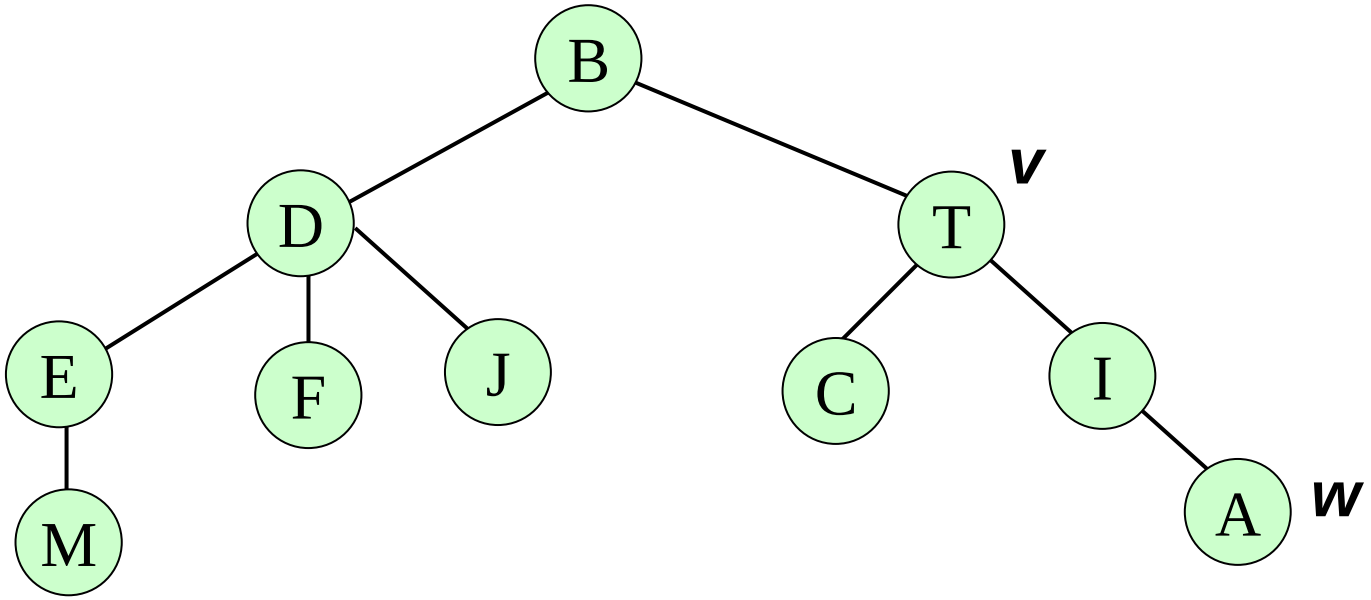
Definições

O número de filhos de um nó *v* é chamado grau de saída de *v*



Definições

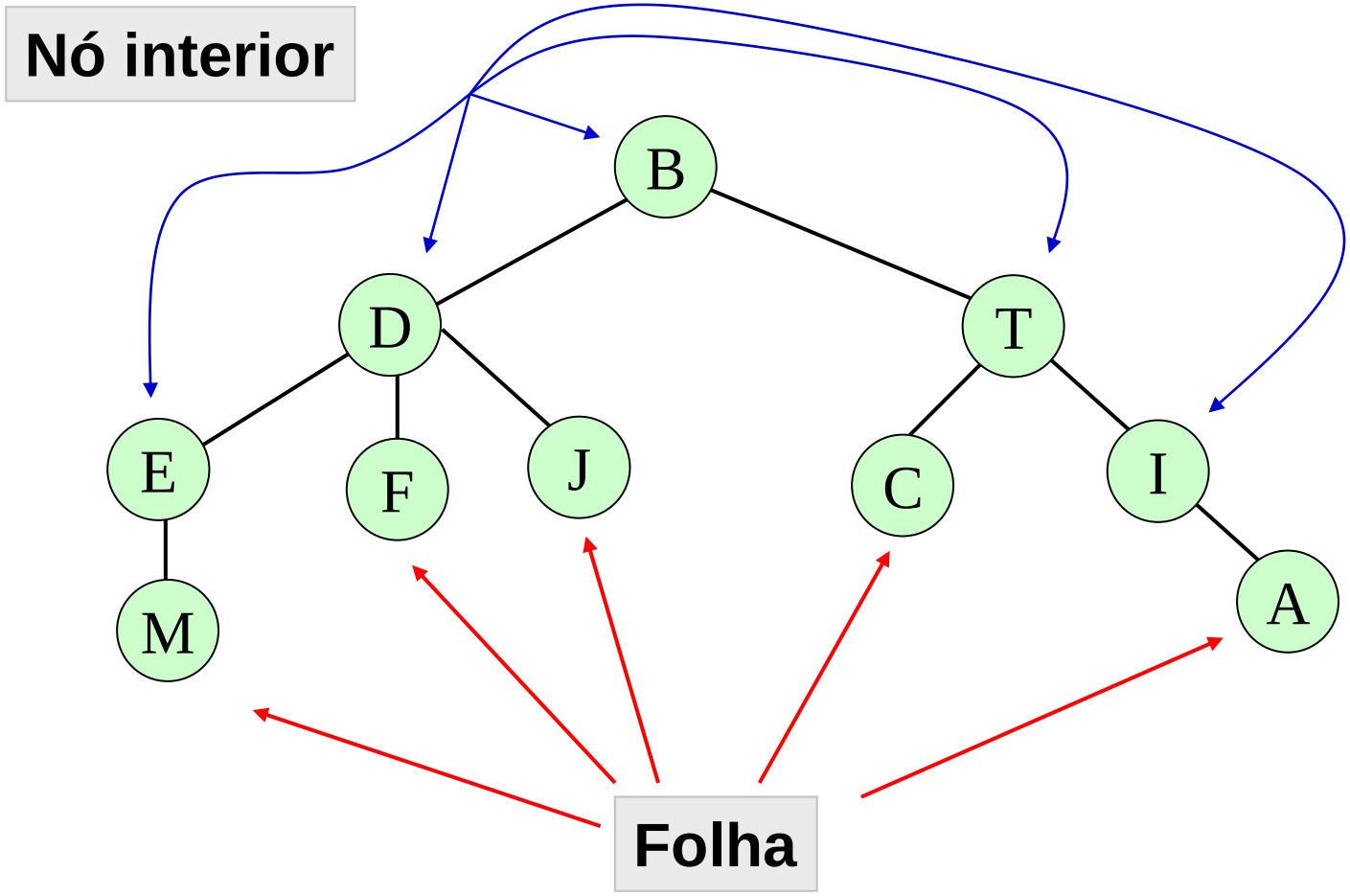
Se *w* pertence a T_v , então *w* é descendente de *v* e *v* é ancestral de *w*



Observação: se *v* ≠ *w*, diz-se ancestral próprio ou descendente próprio

Definições

Um nó com grau de saída igual a 0 é chamado folha



Exercício

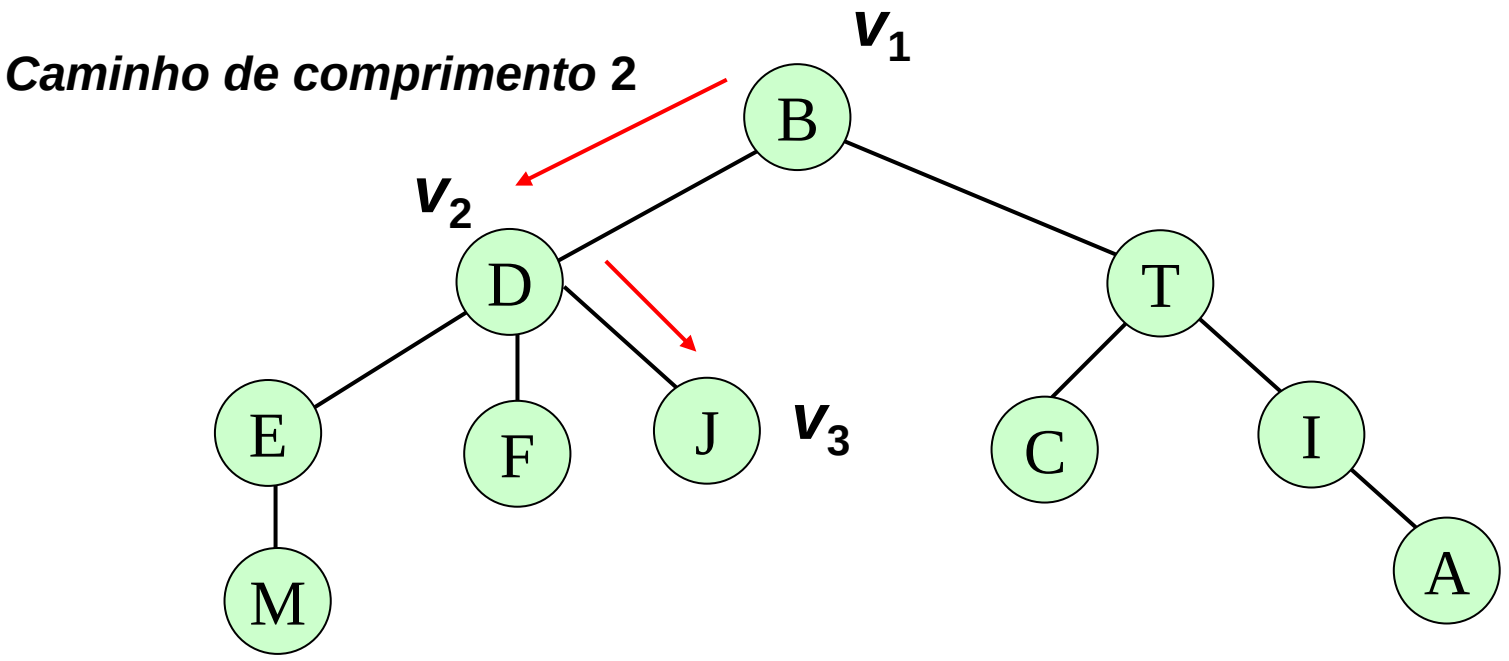
Prove que

Toda árvore com número de nós maior que 1 ($n > 1$) possui no mínimo 1 e no máximo $n - 1$ folhas.



Definições

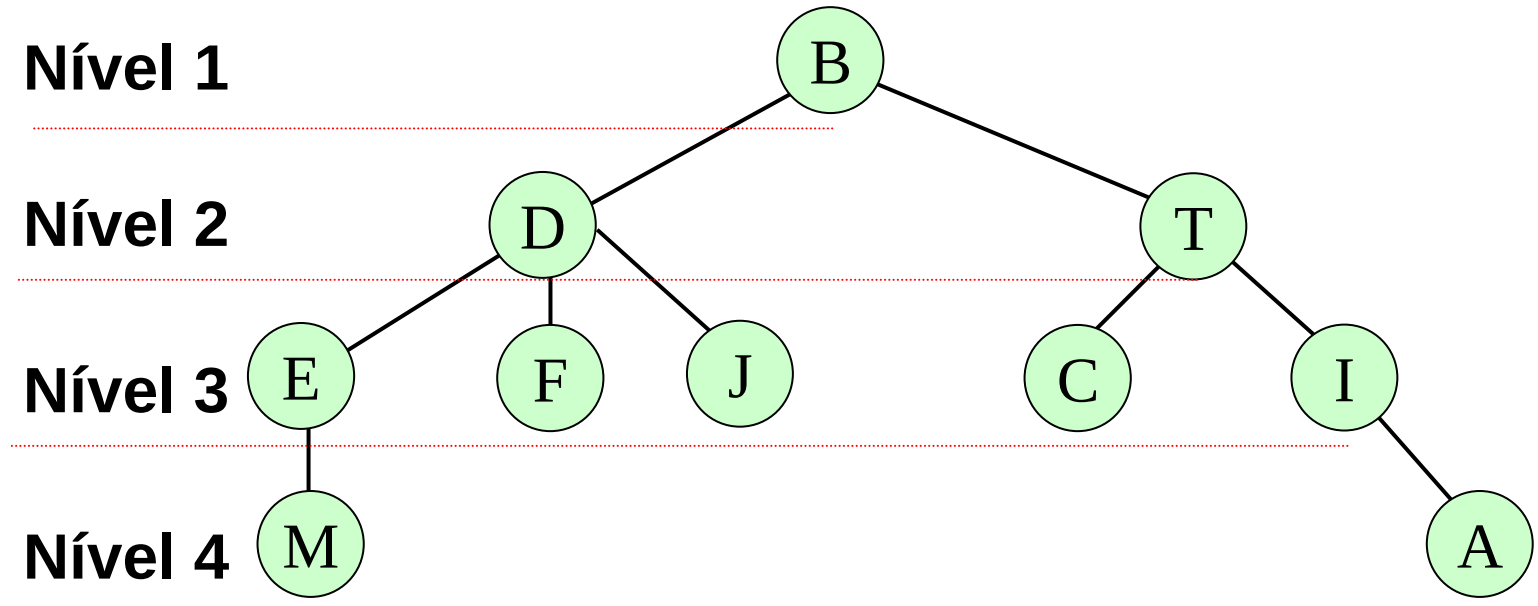
Uma sequência de nós distintos v_1, v_2, ..., v_k tal que para dois nós consecutivos existe sempre a relação *é filho de* (*é pai de*) é dita *um caminho na árvore*.



Comprimento do caminho = k-1

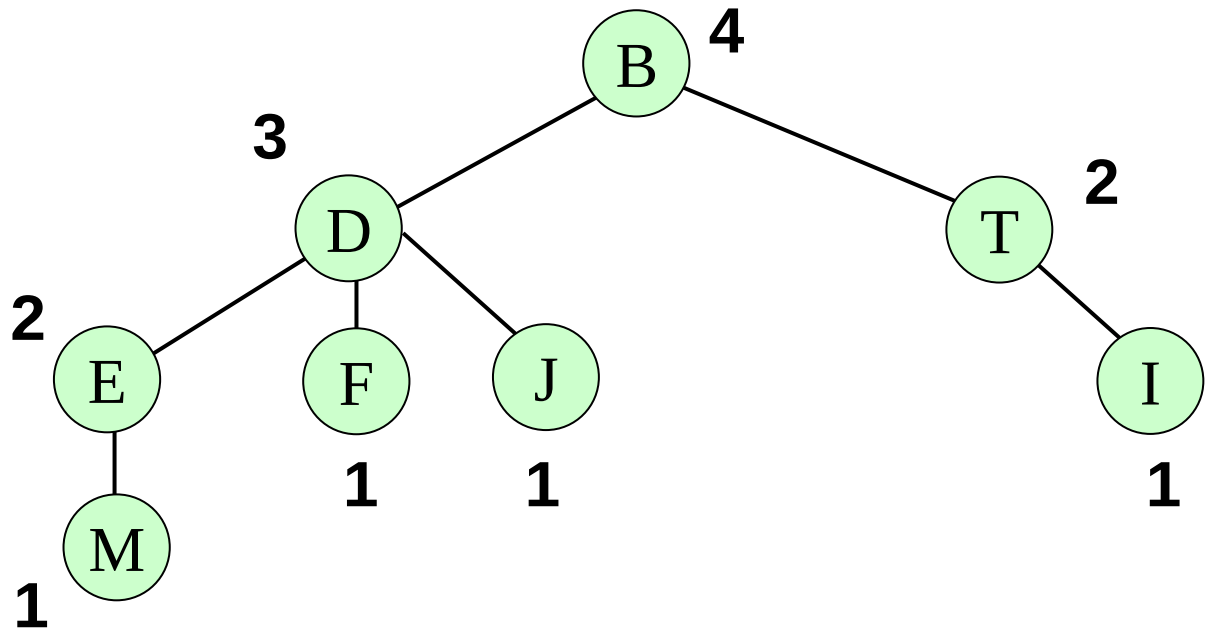
Definições

Nível do nó *v* é o número de nós no caminho entre a raiz da árvore e *v*.



Definições

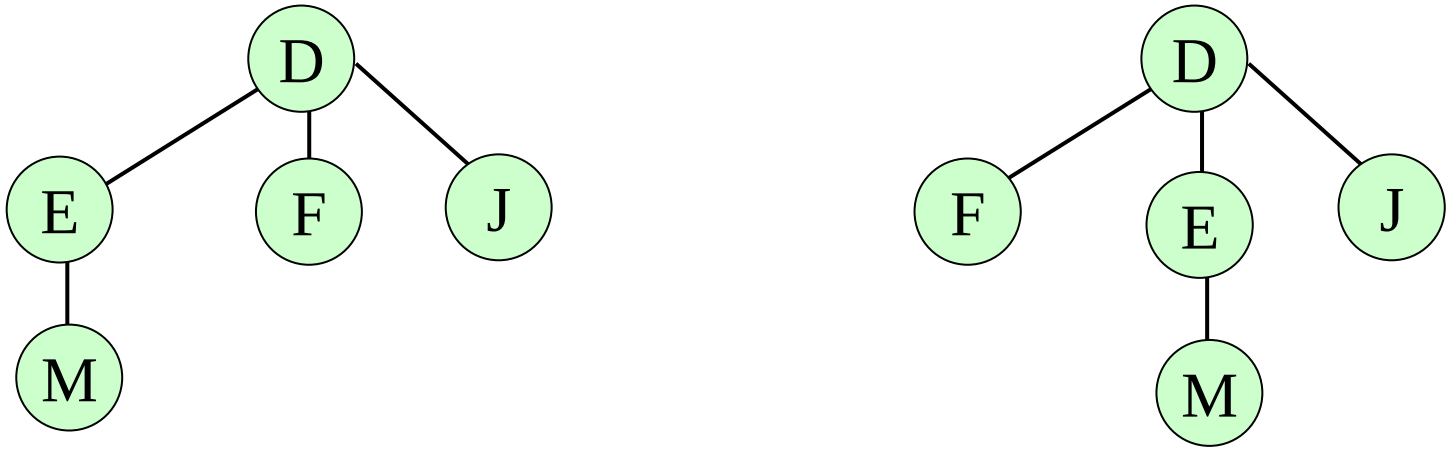
Altura do nó *v* é o número de nós no caminho entre *v* e seu descendente mais distante.



A altura da árvore, h(T), é dada pela altura da raiz (ou pelo nível máximo de seus nós)

Definições

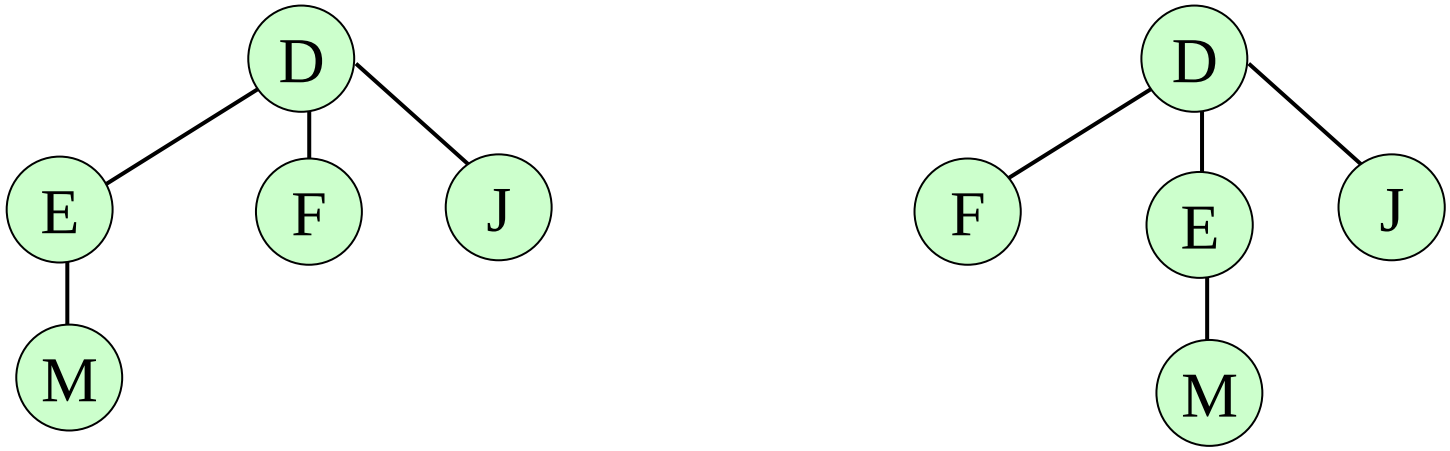
Quando a ordenação dos filhos é considerada, a árvore é dita *ordenada*. Assume-se que a ordenação seja da esquerda para a direita.



Árvores Distintas
(considerando que são ordenadas)

Definições

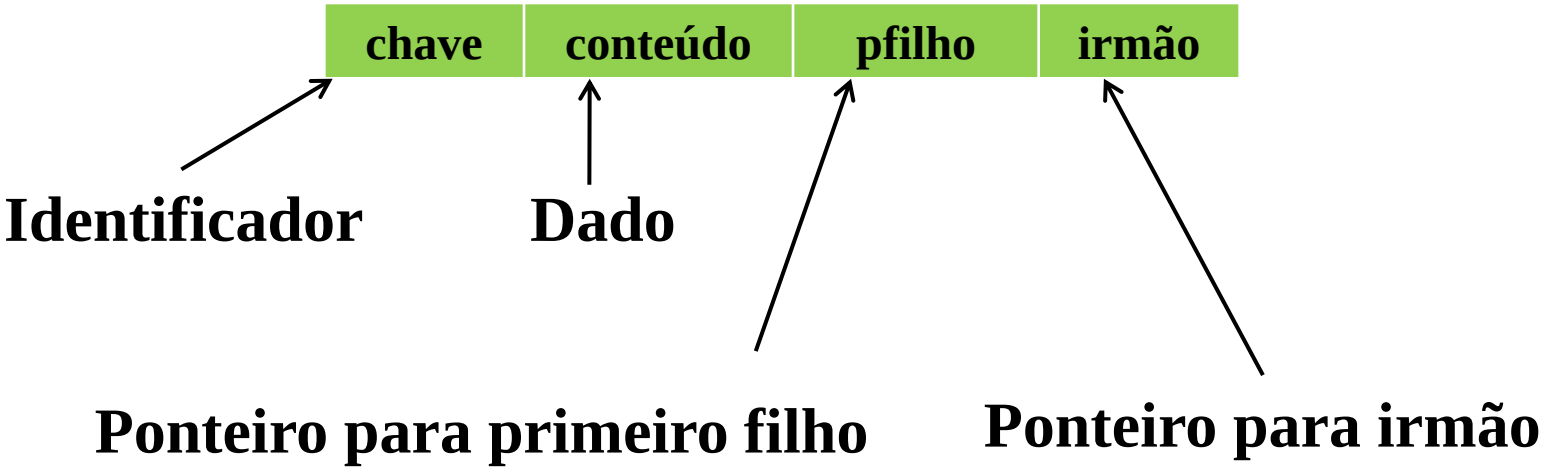
Duas árvores não ordenadas são *isomorfas* quando puderem se tornar coincidentes através de uma permutação na ordem das subárvores de seus nós.



Árvores Isomorfas

Árvore

Nó da árvore



Árvore

Nó da árvore

chave	conteúdo	pfilho	irmão

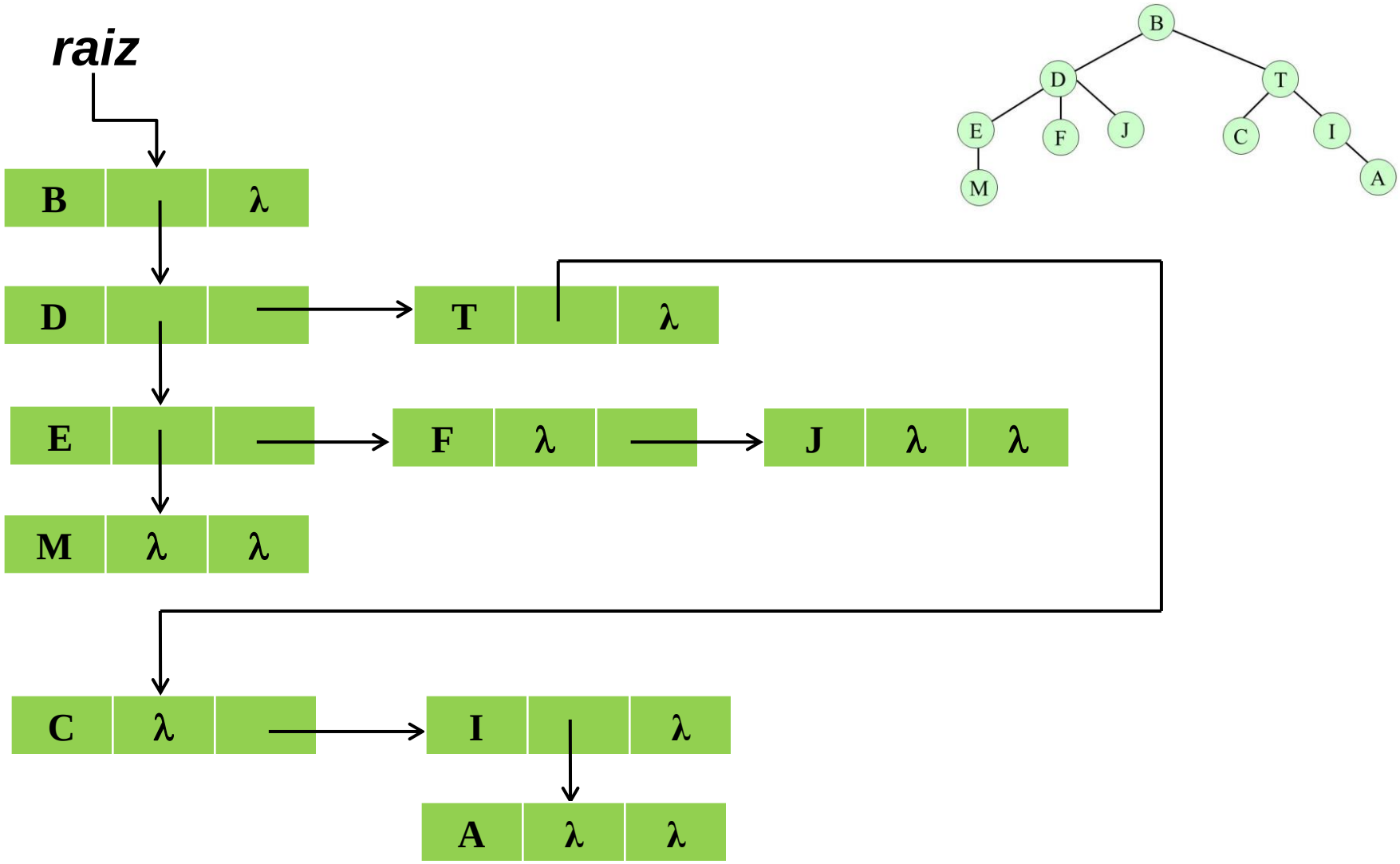
Exemplo C:

```
struct nodo {
  int chave;
  int conteudo;
  struct nodo *pfilho;
  struct nodo *irmao; };
typedef struct nodo no;
```

```
typedef no *pont_no;
```

Observação: O campo *chave* será usado como chave e conteúdo

Árvore



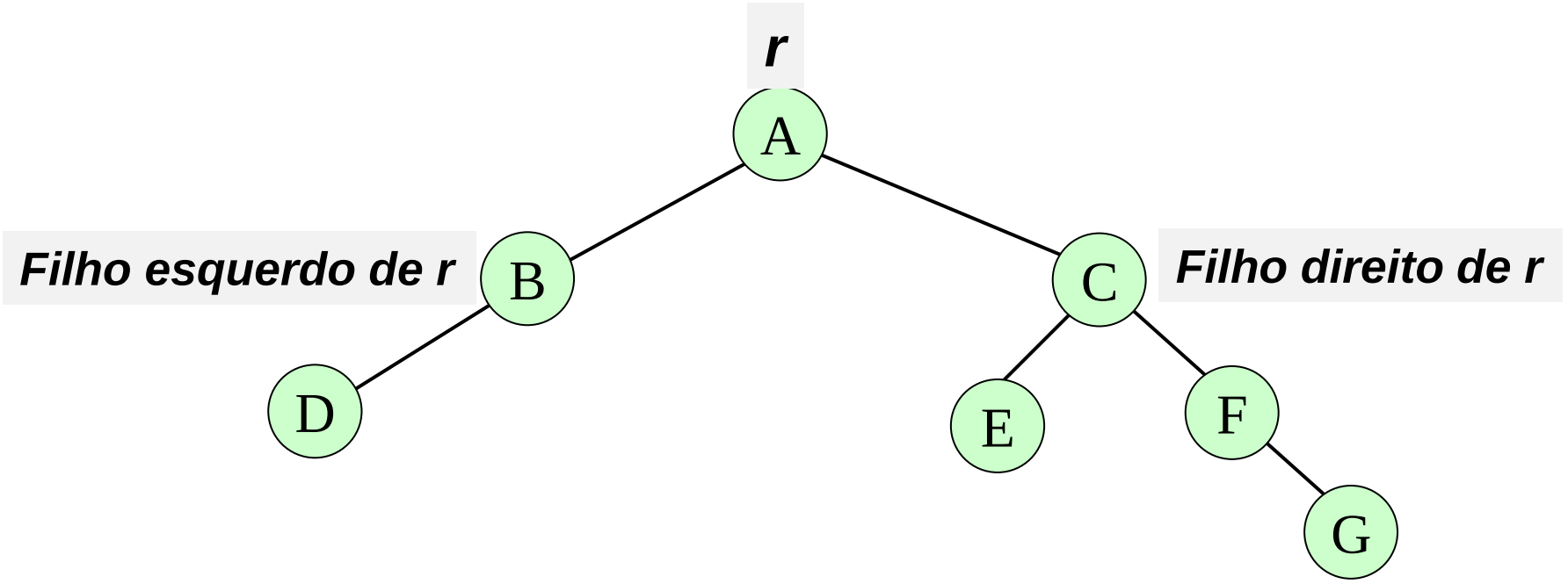
Árvore Binária

Uma árvore binária, *T,* é um conjunto finito de elementos chamados vértices (ou nós), tal que:

***i)T* = ∅ (a árvore é dita vazia) ou**
ii) Existe um nó especial, *r*, dito *raiz* da árvore *T* e os nós restantes são divididos em dois conjuntos disjuntos, a subárvore esquerda e direita de *r*, cada qual uma árvore binária.

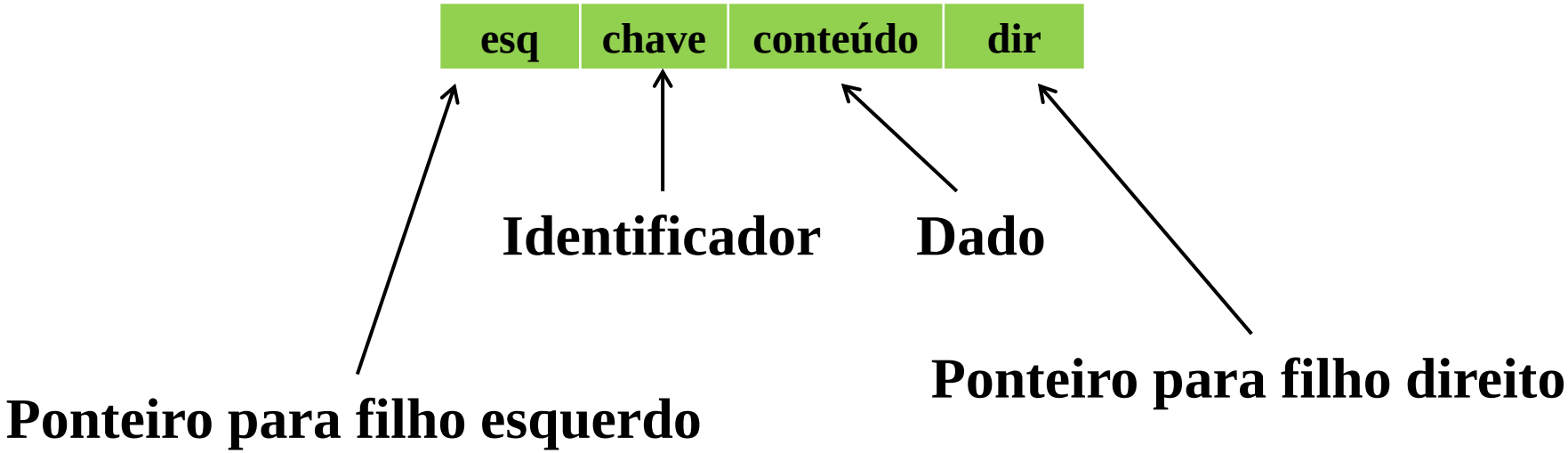


Árvore Binária



Árvore Binária

Nó da árvore



Árvore Binária

Nó da árvore



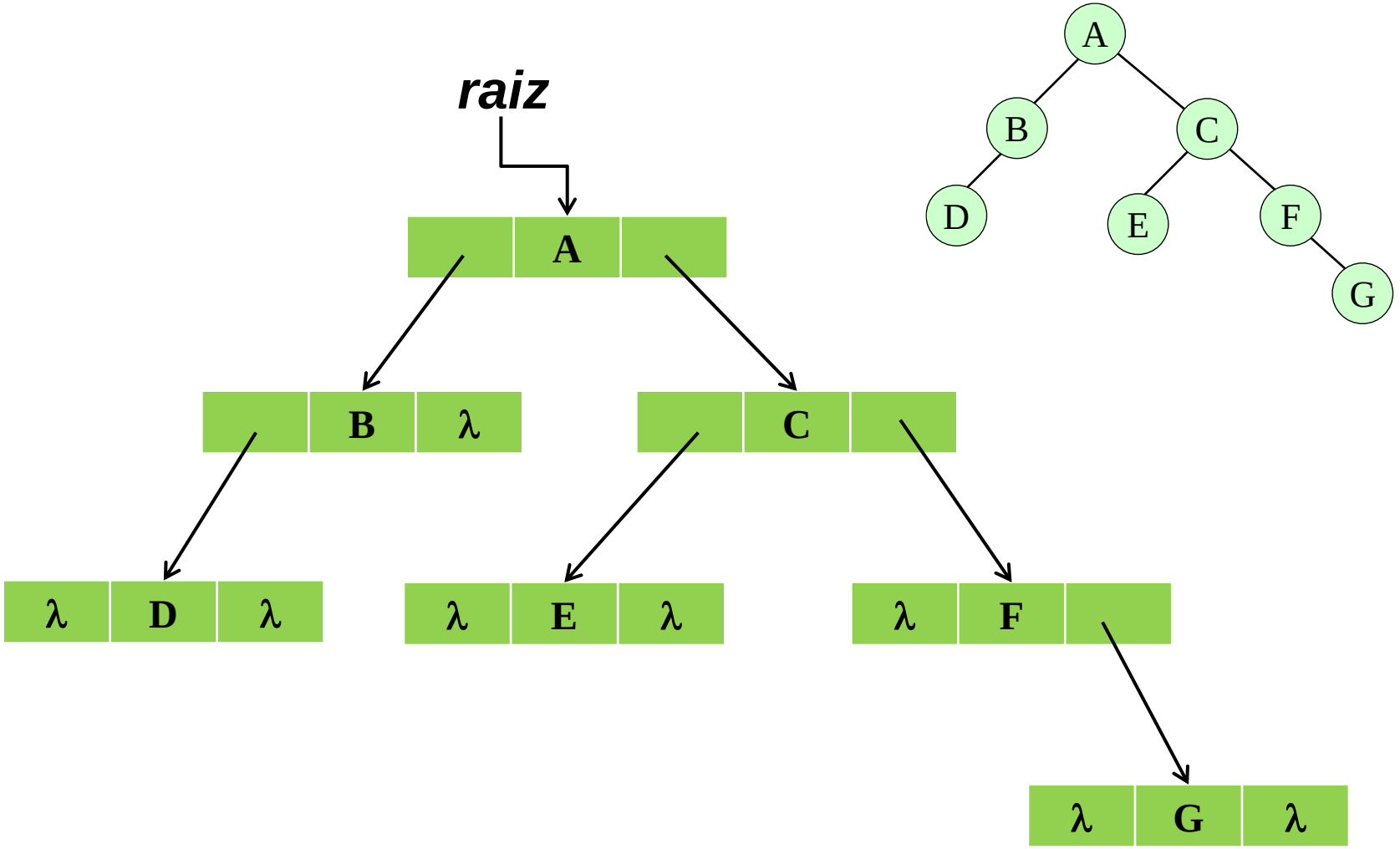
Exemplo C:

```
struct nodo {
  int chave;
  int conteudo;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
```

```
typedef no *pont_no;
```

Observação: O campo *chave* será usado como chave e conteúdo

Árvore Binária



Árvore Binária

Lema 1

O número de subárvores vazias de uma árvore binária com n nós é $n + 1$

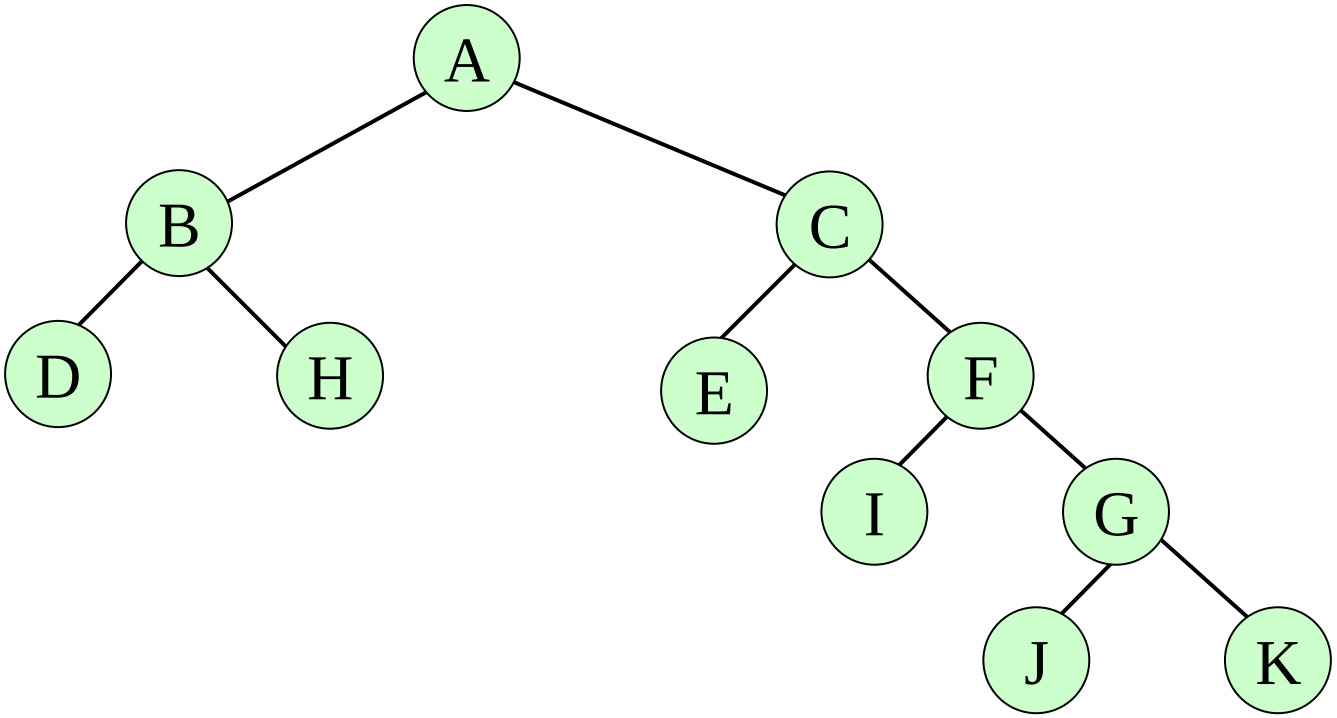
Prova: **Exercício**



Árvores Binárias Especiais

Árvore Estritamente Binária

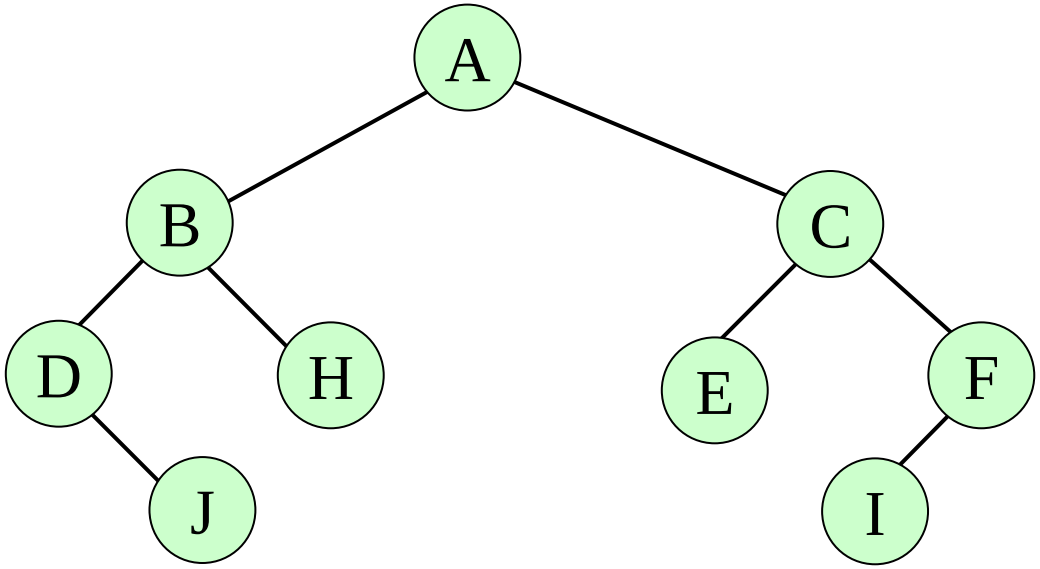
Todo nó possui 0 ou 2 filhos



Árvores Binárias Especiais

Árvore Binária Completa

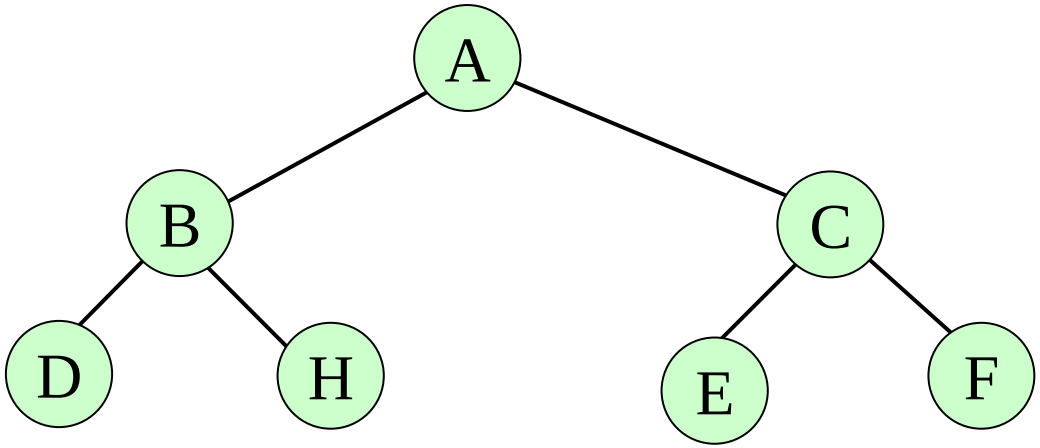
Se *v* é um nó com uma subárvore vazia, então *v* está no último ou no penúltimo nível de *T*.



Árvores Binárias Especiais

Árvore Binária Cheia

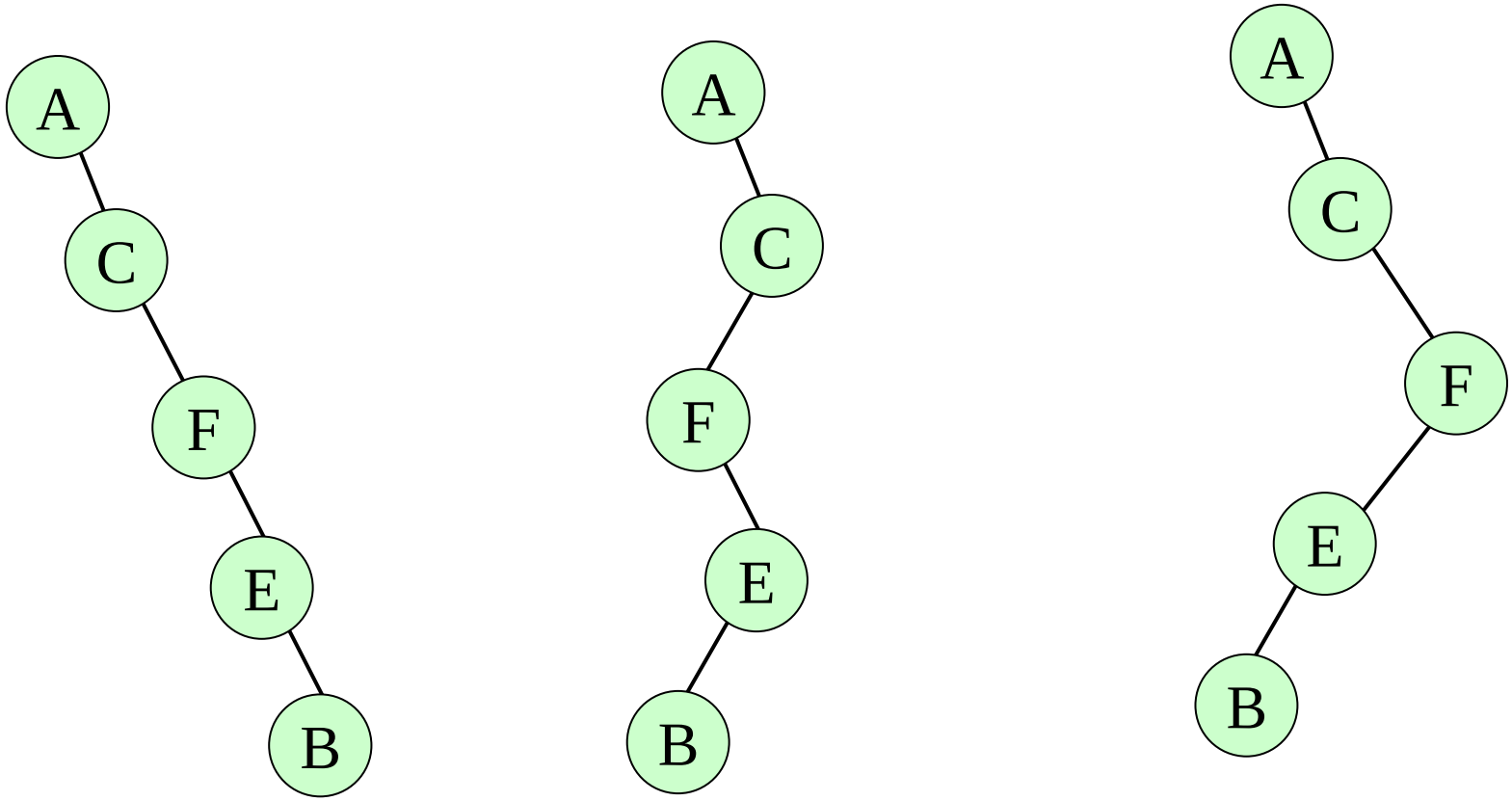
Se *v* é um nó com uma subárvore vazia, então *v* está no último nível de *T*.



Obs. Toda árvore cheia é estritamente binária e completa

Árvores Binárias Especiais

Árvores Ziguezague



Altura máxima para um número fixo de nós

Árvores Binárias Especiais

Lema 2

Considere T uma árvore binária completa com $n > 0$ nós.
Então T possui altura h mínima.
Além disso, $h = 1 + \lfloor \log n \rfloor$.

Prova: **Exercício**



Árvores Binárias Especiais

Lema 3

Seja T uma árvore binária completa com n nós e altura h.
Então, $2^{h-1} \leq n \leq 2^h - 1$

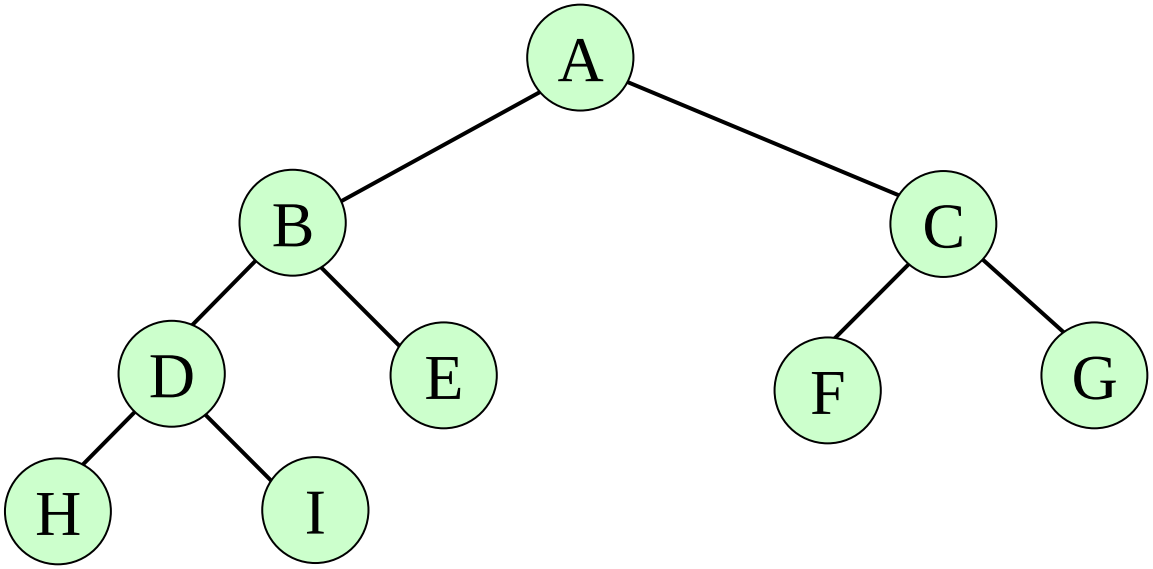
Prova: **Exercício**



Árvores Binárias Especiais

Representação Vetorial de uma Árvore Binária Completa

Caso especial onde o último nível é preenchido da esquerda para a direita



***n*= 9** ***max* = 15**

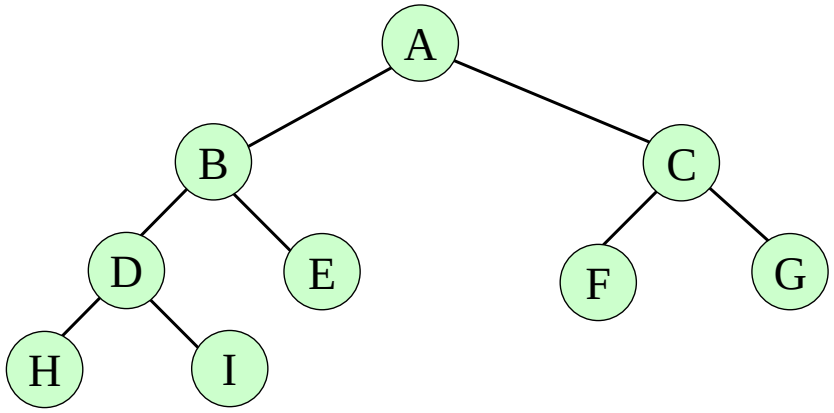
A	B	C	D	E	F	G	H	I						
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Árvores Binárias Especiais

Representação Vetorial de uma Árvore Binária Completa

***n*= 9**

A	B	C	D	E	F	G	H	I						
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Filho esquerdo do nó na posição *i* :
posição 2*i*

Filho direito do nó na posição *i* :
posição 2*i* + 1

Pai do nó na posição *i* :
posição $\lfloor i/2 \rfloor$

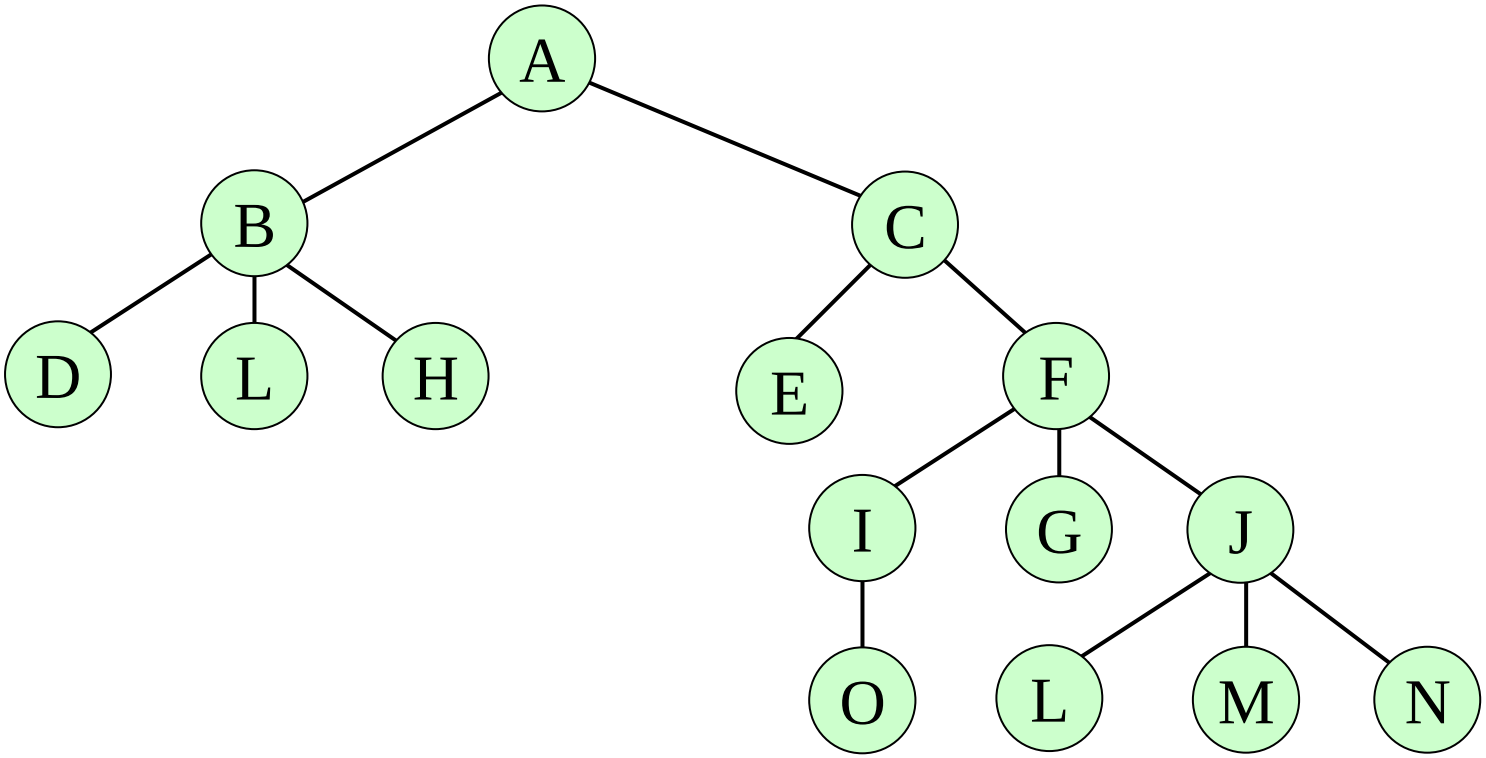
Árvore *m*-ária

Uma árvore *m*-ária, *T*, *m* ≥ 2, é um conjunto finito de elementos chamados vértices (ou nós), tal que:

i) ***T* = ∅ (a árvore é dita vazia) ou**

ii) Existe um nó especial, *r*, dito *raiz* da árvore *T* e os nós restantes são divididos em *m* conjuntos disjuntos, as *m* subárvores de *r*, cada qual uma árvore *m*-ária.

Árvore ternária



Árvore *m*-ária

Exercício

Definir:

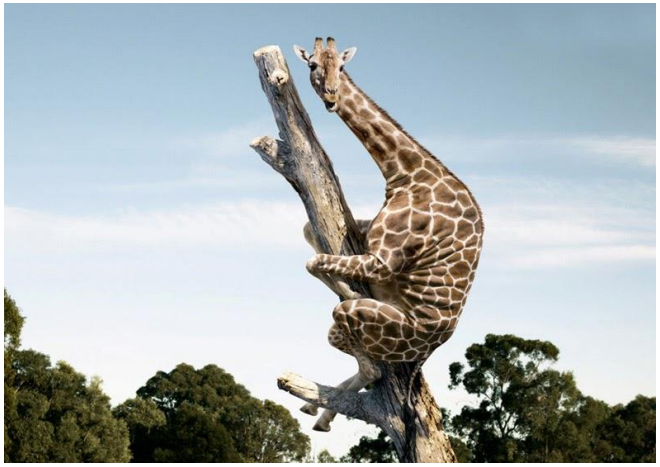
1) **Árvore estritamente *m*-ária**
2) **Árvore *m*-ária completa**
3) **Árvore *m*-ária cheia**



Percurso em Árvore Binária

Um percurso é uma visita sistemática a cada nó da árvore.

Corresponde a conhecer a informação contida no nó, percorrer a subárvore esquerda e percorrer a subárvore direita.



Percurso em Árvore Binária

Considerando da esquerda para a direita

Considerando que visitar(raiz) significa imprimir o conteúdo do nó.

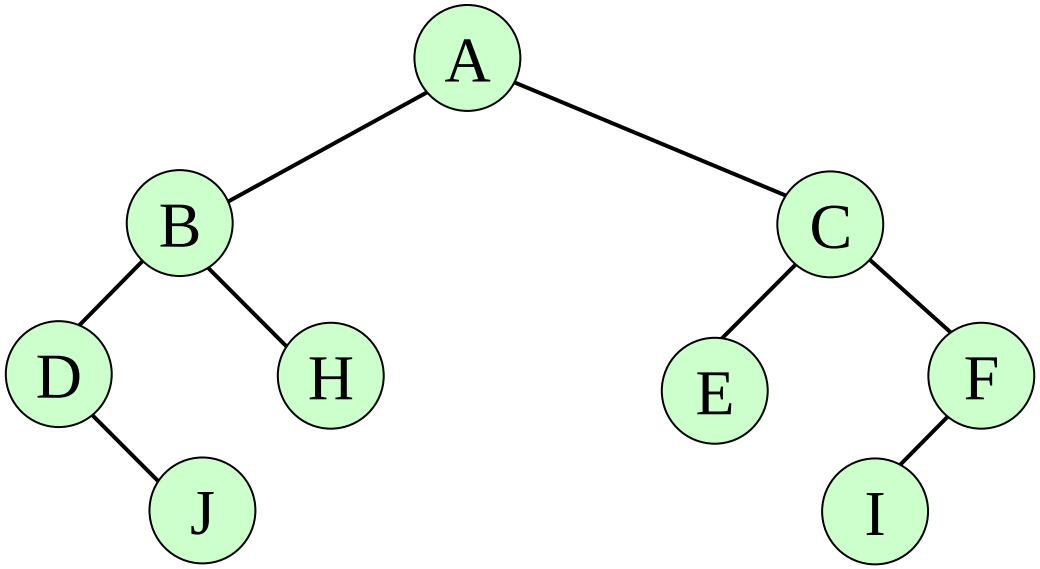
Pré-ordem

visitar(raiz)
percorrer subárvore esquerda da raiz em pré-ordem
percorrer subárvore direita da raiz em pré-ordem

Percurso em Árvore Binária

Exemplo: Pré-ordem

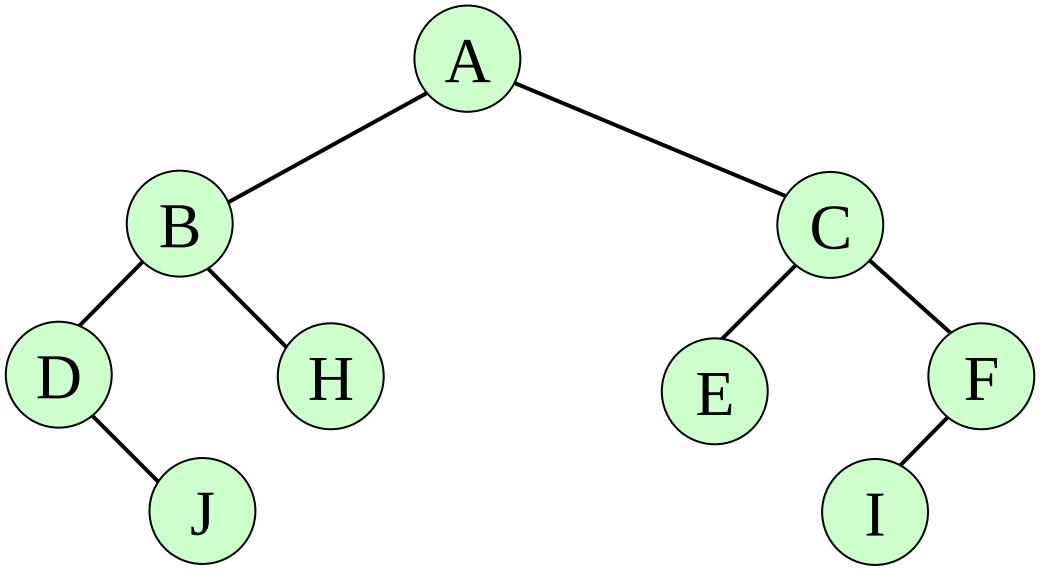
visitar(raiz)
percorrer subárvore esquerda da raiz em pré-ordem
percorrer subárvore direita da raiz em pré-ordem



Percurso em Árvore Binária

Exemplo: Pré-ordem

visitar(raiz)
percorrer subárvore esquerda da raiz em pré-ordem
percorrer subárvore direita da raiz em pré-ordem



Pré-ordem: **A B D J H C E F I**

Percurso em Árvore Binária

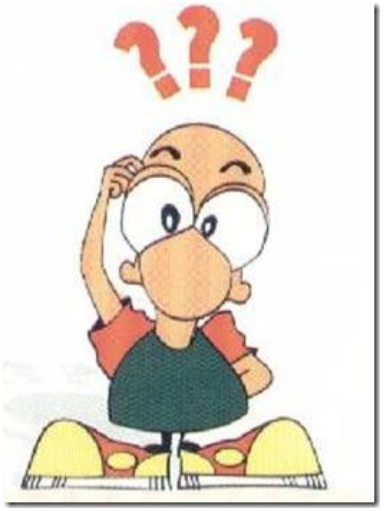
Principal

```
se pt ≠ λ
    pre_ordem(pt)
```

Algoritmo pre_ordem(pont_no *pt*)

```
visitar(pt)
se pt↑.esq ≠ λ
    pre_ordem(pt↑.esq)
se pt↑.dir ≠ λ
    pre_ordem(pt↑.dir)
```

Percurso em Árvore Binária



Exercício

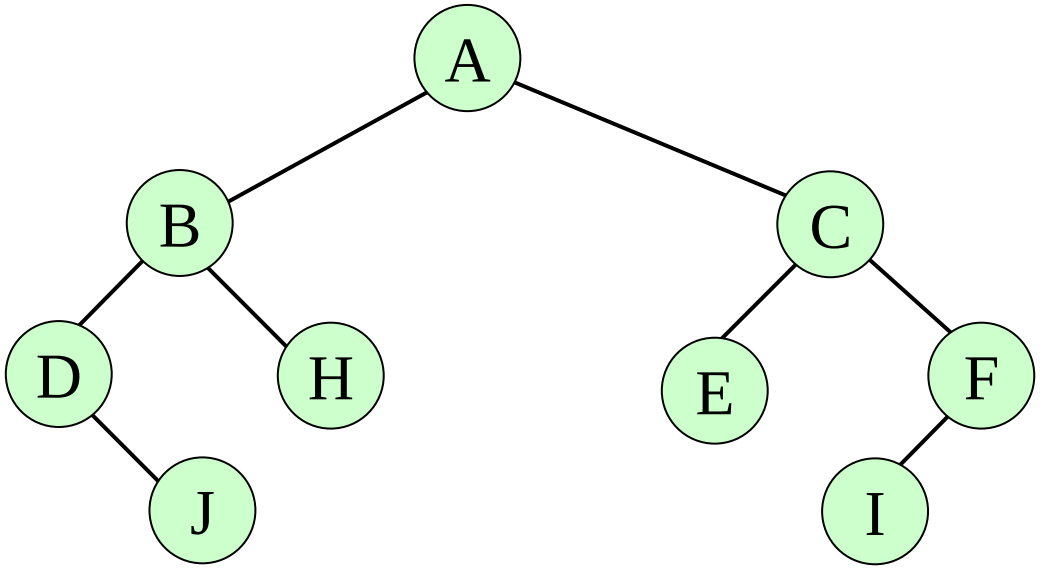
Qual a complexidade de pre_ordem ( )?



Percurso em Árvore Binária

Ordem Simétrica

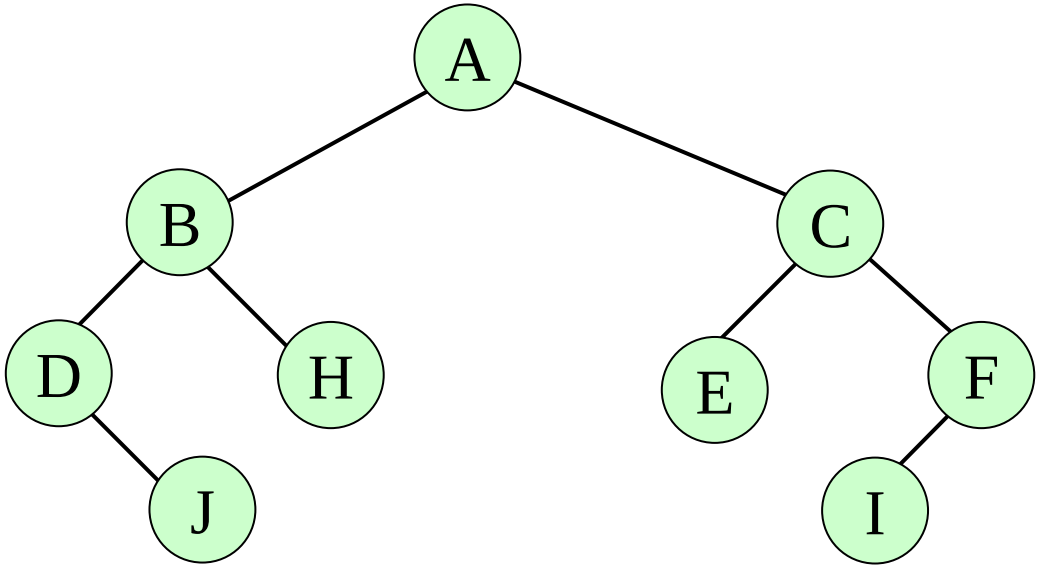
percorrer subárvore esquerda da raiz em ordem simétrica
visitar(raiz)
percorrer subárvore direita da raiz em ordem simétrica



Percurso em Árvore Binária

Ordem Simétrica

percorrer subárvore esquerda da raiz em ordem simétrica
visitar(raiz)
percorrer subárvore direita da raiz em ordem simétrica

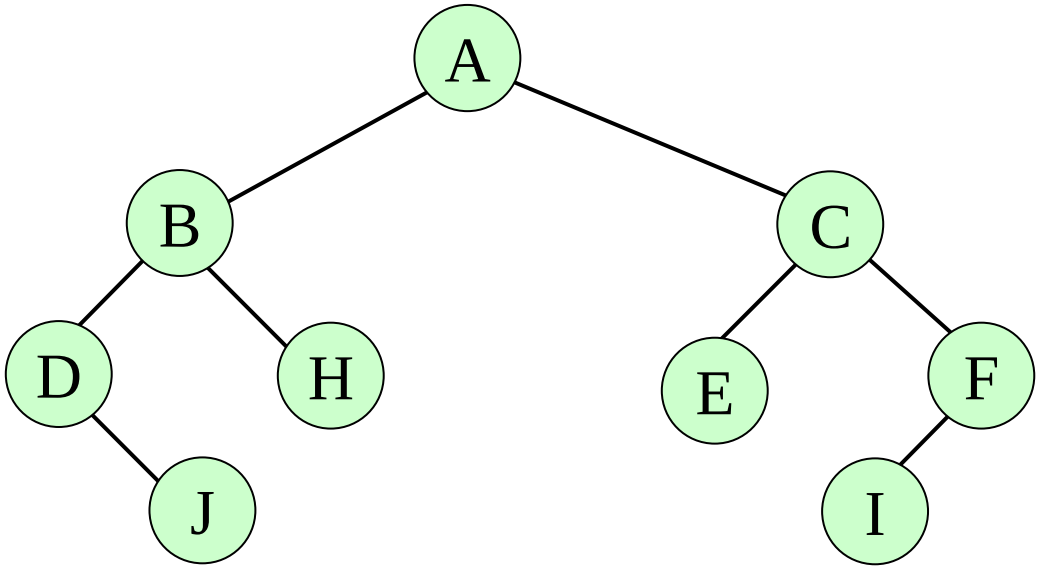


Ordem Simétrica: D J B H A E C I F

Percurso em Árvore Binária

Pós ordem

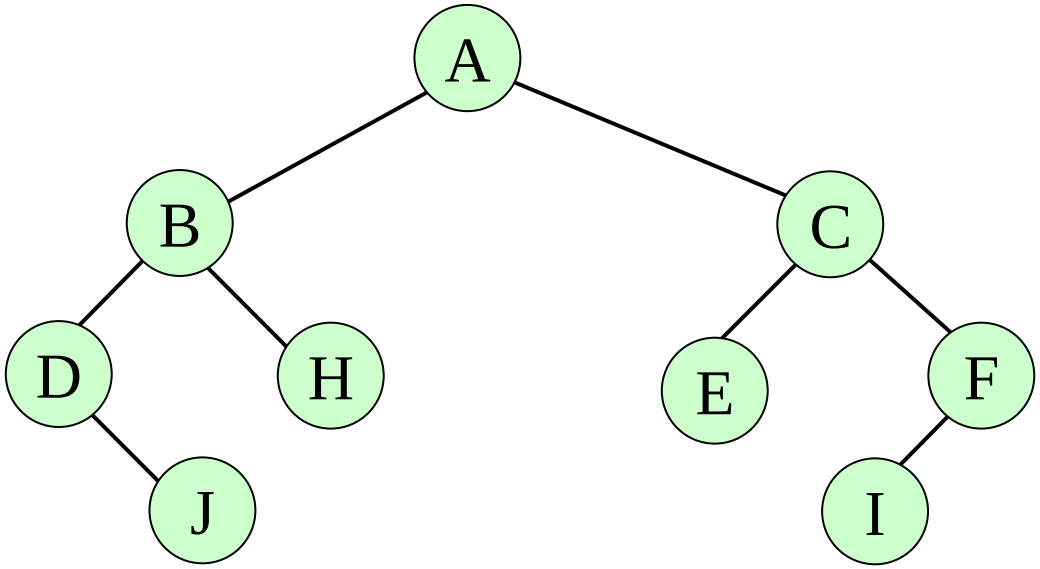
percorrer subárvore esquerda da raiz em pós-ordem
percorrer subárvore direita da raiz em pós-ordem
visitar(raiz)



Percurso em Árvore Binária

Pós ordem

percorrer subárvore esquerda da raiz em pós-ordem
percorrer subárvore direita da raiz em pós-ordem
visitar(raiz)



Pós-Ordem: J D H B E I F C A

Percurso em Árvore Binária



Exercício

Fazer as versões iterativas de:
pre_ordem
ordem_simetrica
pos_ordem



Exercício

Fazer um algoritmo de percurso pré-ordem para uma árvore qualquer com a representação apresentada.
Faça a versão recursiva e a versão iterativa.



Exercício Aula

Faça um algoritmo para calcular a altura de todos os nós de uma árvore binária

Exercício Aula

Nó da árvore

Exemplo C:

```
struct nodo {
  int chave;
  int altura;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
```



```
typedef no *pont_no;
```

Exercício Aula

Calcula Altura

Principal

```
se pt ≠ λ
    pos_ordem(pt)
```

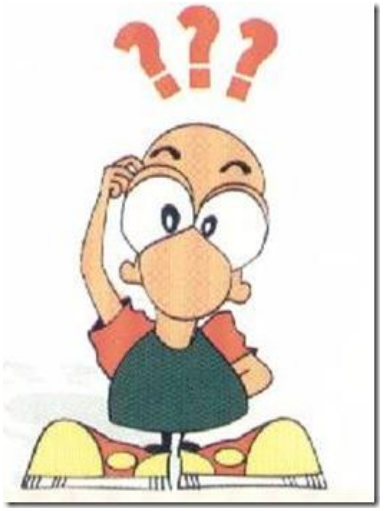
pos_ordem(pont_no *pt*)

```
se (pt↑.esq ≠ λ)
    pos_ordem(pt↑.esq)
se (pt↑.dir ≠ λ)
    pos_ordem(pt↑.dir)
visitar(pt)
```

visita(pont_no *pt*)

```
se (pt↑.esq = λ)
   alt1 ← 0
senão alt1 ← pt↑.esq↑.altura
se (pt↑.dir = λ)
   alt2 ← 0
senão alt2 ← pt↑.dir↑.altura
se (alt1 > alt2)
    pt↑.altura ← alt1 + 1
senão
    pt↑.altura ← alt2 + 1
```

Árvore Binária



Exercício

Qual a complexidade do algoritmo para calcular a altura de todos os nós da árvore binária?



Percurso em Árvore Binária

Percurso em Nível

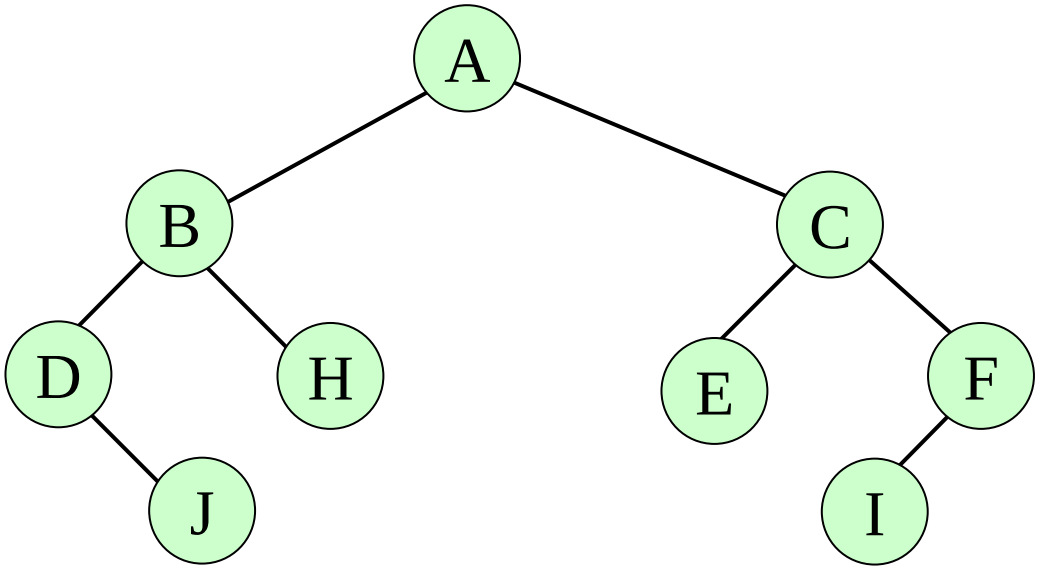
Considerando da esquerda para a direita

Considerando que visitar(raiz) significa imprimir o conteúdo do nó.

Imprime os nós de cada nível da árvore

Percurso em Árvore Binária

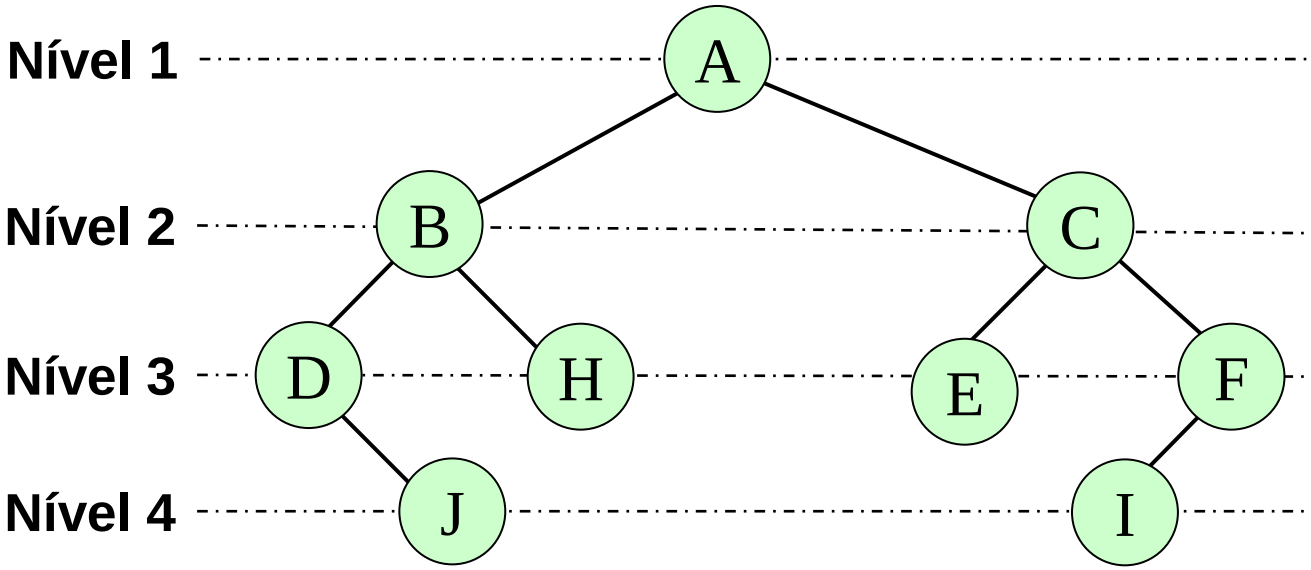
Percurso em Nível



Nível: A B C D H E F J I

Exercício Aula

Exercício: Faça um algoritmo de percurso em nível de uma árvore binária



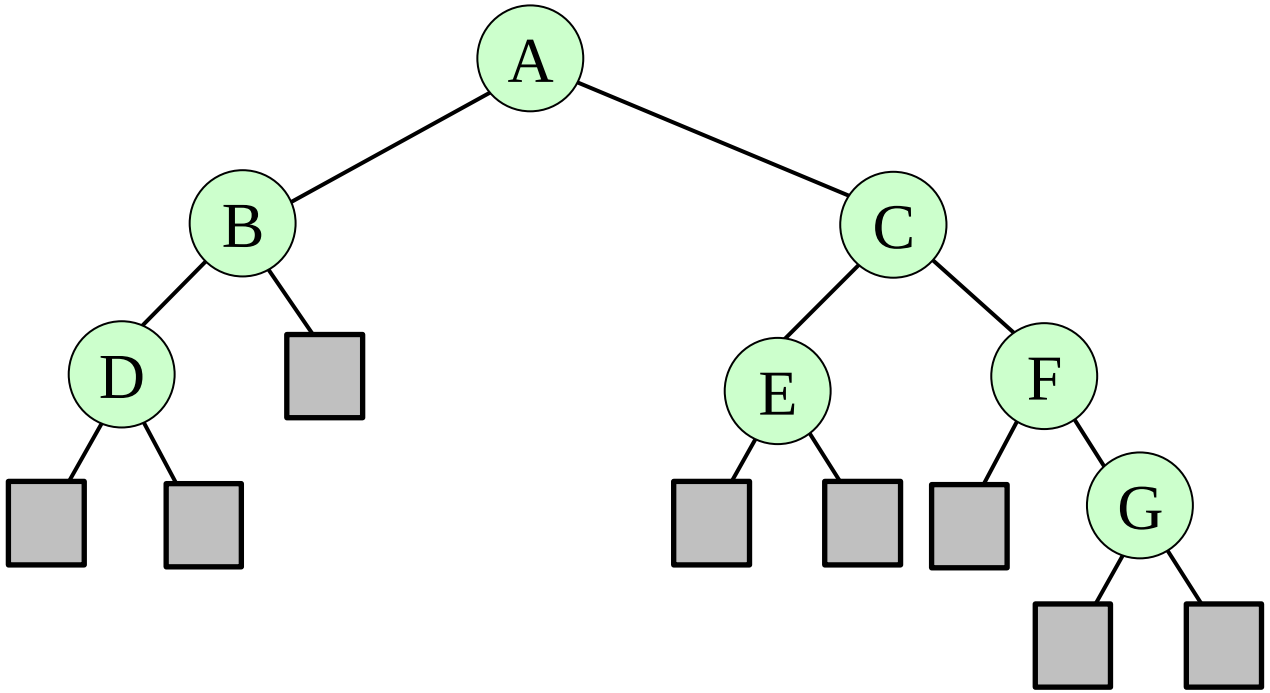


```
buscaLargura (pt)
        fila filaNos;
        se pt != NULL
                filaNos.inserir (pt)
                while (!filaNos.vazio())
                        nox = filaNos.remover()
                        visitar (nox)
                        se nox.esq != NULL
                                filaNos.inserir(nox.esq)
                        se nox.dir != NULL
                                filaNos.inserir(nox.dir)
```

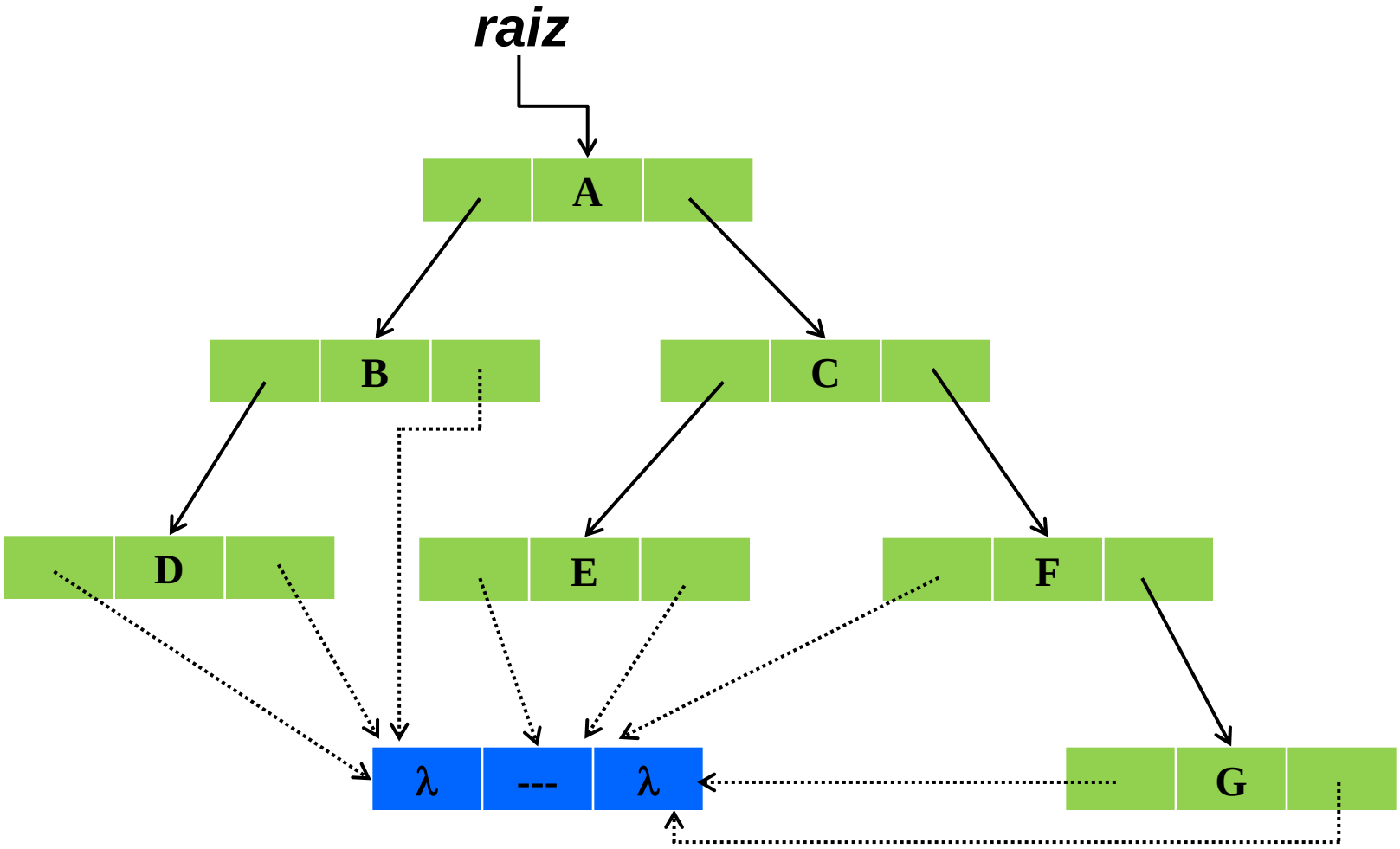
Árvore Binária Estendida

Cada filho vazio (ponteiro nulo) é substituído por um nó *externo*.

Os nós da árvore são nós *internos*.



Árvore Binária Estendida



Árvore Binária Estendida

Lema 4

Uma árvore binária com n nós internos possui $n + 1$ nós externos

Prova: A prova é feita por indução em n.

Caso base: $n = 0$
Então existe 1 nó externo, o lema vale trivialmente.

Supor $n \geq 1$ e que a hipótese vale para toda árvore com menos de n nós.

A árvore tem uma raiz e outros n-1 nós divididos na subárvore esquerda e direita da raiz.

Árvore Binária Estendida

Lema 4

Uma árvore binária com n nós internos possui $n + 1$ nós externos

Prova(cont)

Suponha que k nós internos estão na subárvore esquerda da raiz e $(n – 1) – k$ nós estão na subárvore direita da raiz.

Ambas subárvores são binárias estendidas e possuem menos de n nós. Portanto o lema vale trivialmente para elas.

Assim, a subárvore esquerda possui $k + 1$ nós externos e a subárvore direita possui $(n -1)+k+1 = n - k$ nós externos.

Então a árvore com n nós internos possui $(k + 1) + n – k = n + 1$ nós externos.

Árvore Binária com Costura

Motivação

Utilizar os ponteiros nulos para conter informação relevante.

Considerando um percurso:

- Um ponteiro *esq* nulo passa a apontar para o seu antecessor no percurso

- Um ponteiro *dir* nulo passa a apontar para o seu sucessor no percurso

Obs: Precisa diferenciar um ponteiro para filho de uma costura.

Árvore Binária com Costura

Nó da árvore



Exemplo C:

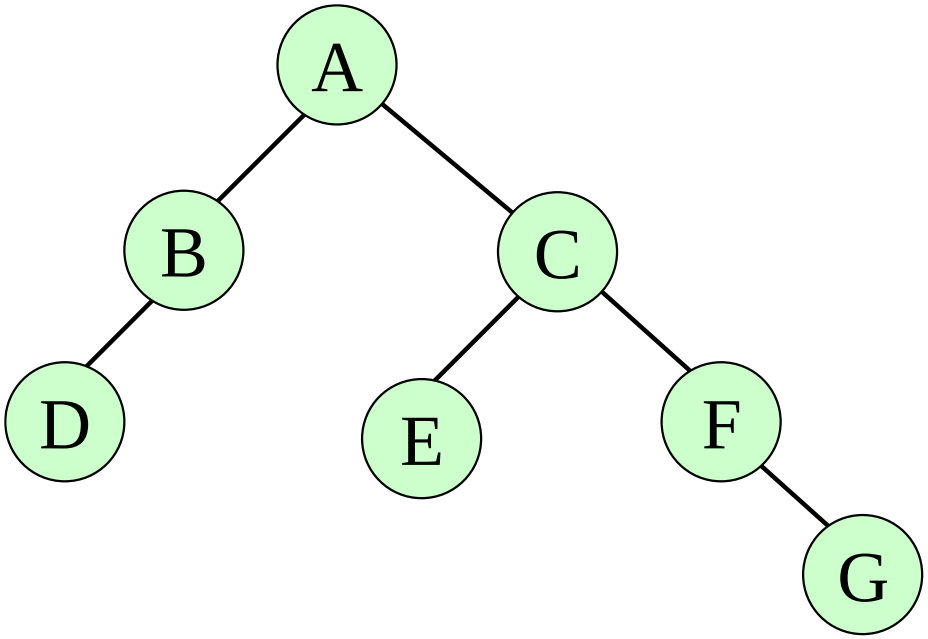
```
struct nodo {
  int chave;
  char ecos;
  char dcos;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
```

Campos de diferenciação
0 – não é costura
1 – é costura

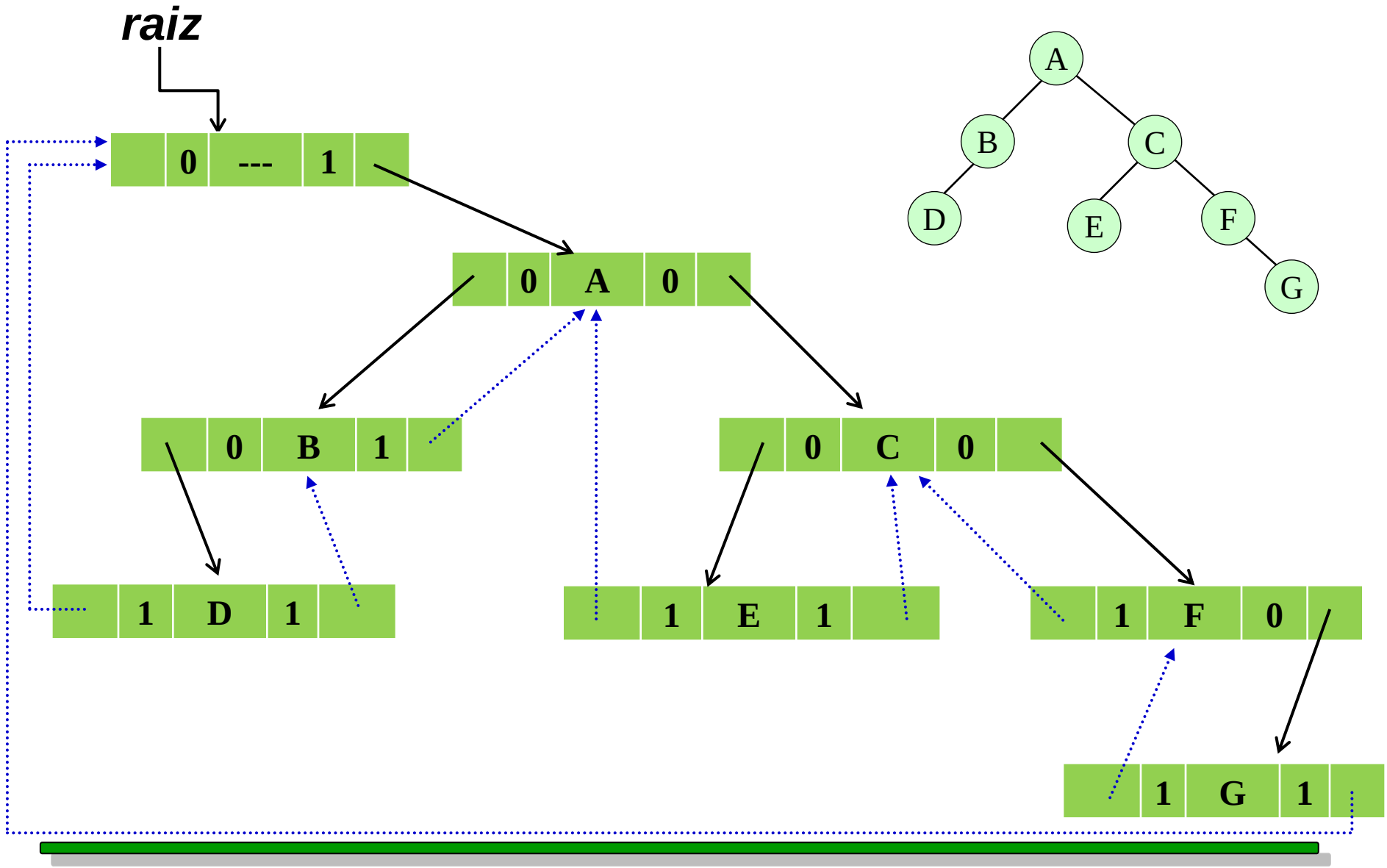
```
typedef no *pont_no;
```

Árvore Binária com Costura

Como esta árvore ficaria representada com costura segundo um percurso em ordem simétrica?



Árvore Binária com Costura



Exercício Aula

Considerando as costuras faça um algoritmo para obter o sucessor de um nó em um percurso em ordem simétrica.

Exercício Aula

```
sucessor(pont_no pt1, pont_no pt2)

   pt2 ← pt1↑.dir
   se (pt1↑.dcos = 0)
        enquanto (pt2↑.ecos = 0)
             pt2 ← pt2↑.esq
```

Exercício

Considerando as costuras faça:

1. **Um algoritmo para obter o antecessor de um nó em um percurso em ordem simétrica.**
2. **Um algoritmo de percurso em ordem simétrica.**



Exercício

Prove que

Toda árvore com número de nós maior que 1 (n > 1) possui no mínimo 1 e no máximo n – 1 folhas.

O número de subárvores vazias de uma árvore binária com n nós é $n + 1$

Considere T uma árvore binária completa com $n > 0$ nós.
Então T possui altura h mínima.
Além disso, $h = 1 + \lfloor \log n \rfloor$.

Exercício

Definir:

1) **Árvore estritamente *m*-ária**
2) **Árvore *m*-ária completa**
3) **Árvore *m*-ária cheia**

Analise o algoritmo recursivo pre_ordem ( )

Fazer as versões iterativas de:

pre_ordem
ordem_simetrica
pos_ordem

Exercício

Qual a complexidade do algoritmo para calcular a altura de todos os nós da árvore binária?

Considerando as costuras faça:

1. **Um algoritmo para obter o antecessor de um nó em um percurso em ordem simétrica.**
2. **Um algoritmo de percurso em ordem simétrica.**

Exercício

Lema 4

Uma árvore binária com n nós internos possui $n + 1$ nós externos

Solução: A prova é feita por indução em n.

Caso base: $n = 0$
Então existe 1 nó externo, o lema vale trivialmente.

Supor $n \geq 1$ e que a hipótese vale para toda árvore com menos de n nós.

A árvore tem uma raiz e outros n-1 nós divididos na subárvore esquerda e direita da raiz.

Exercício

Lema 4

Uma árvore binária com n nós internos possui $n + 1$ nós externos

Suponha que k nós internos estão na subárvore esquerda da raiz e $(n - 1) - k$ nós estão na subárvore direita da raiz.

Ambas subárvores são binárias estendidas e possuem menos de n nós. Portanto o lema vale trivialmente para elas.

Assim, a subárvore esquerda possui $k + 1$ nós externos e a subárvore direita possui $(n - 1) + k + 1 = n - k$ nós externos.

Então a árvore com n nós internos possui $(k + 1) + n - k = n + 1$ nós externos. ■

Grafo Simples

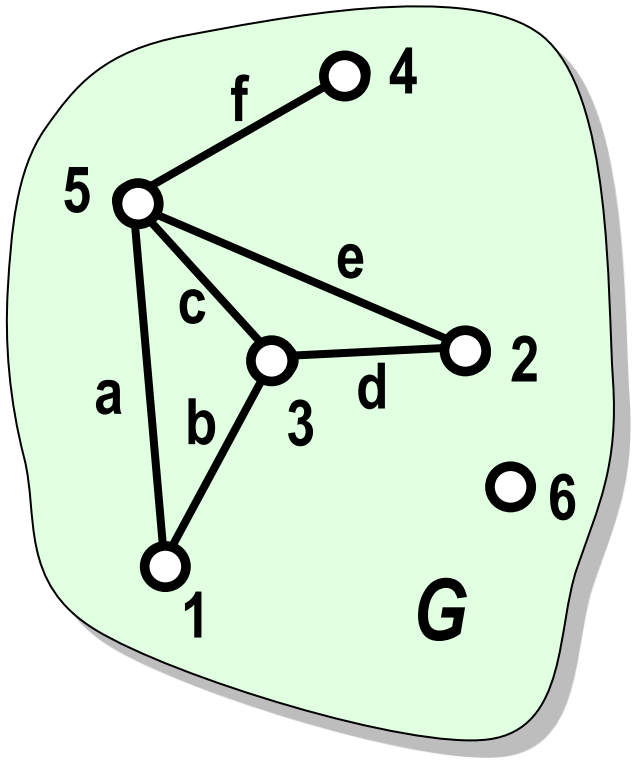
Um grafo $G = (N,M)$ é uma dupla de conjuntos finitos N e M, tal que cada elemento de M define uma relação entre exatamente dois elementos distintos de N e não existem dois elementos distintos de M tais que definam a mesma relação entre o mesmo par de elementos de N.

Grafo Simples

Exemplo:

$N = \{1, 2, 3, 4, 5, 6\}$

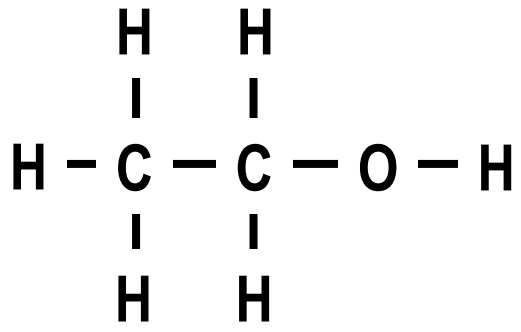
$M = \{a, b, c, d, e, f\}$

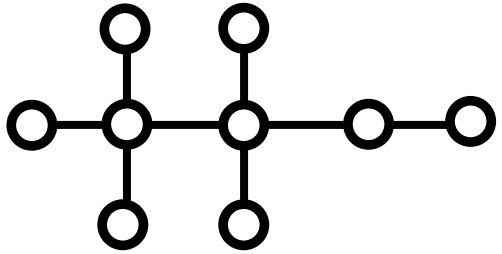


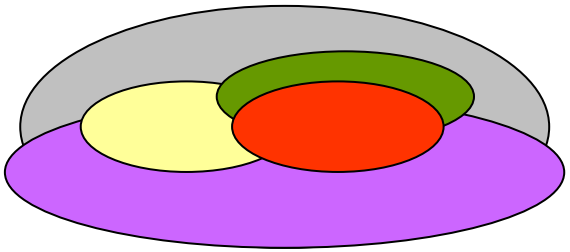
$I = \{(1,5), (1,3), (3,5), (2,3), (2,5), (4,5)\}$

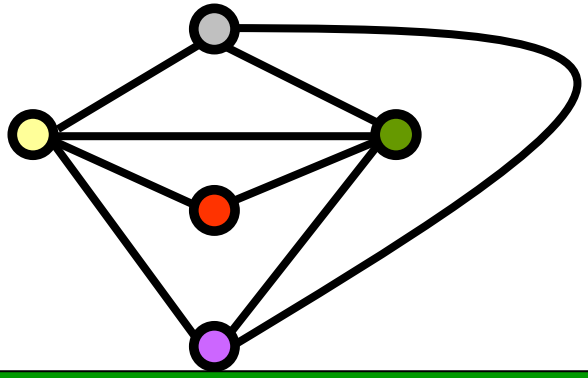
Grafo Simples

Exemplo:









Árvore

Teorema de Caracterização de Árvores

1. ***G* é uma árvore**
2. **Existe exatamente um caminho entre cada par de vértices de *G***
3. ***G* é conexo e m=n-1**
4. ***G* é acíclico e m=n-1**
5. ***G* é acíclico e se quaisquer dois vértices não adjacentes de *G* forem conectados por uma aresta, então o grafo resultante conterá exatamente um ciclo.**